S32K3XX Real Time Control: eMIOS Public

Automotive Systems & Applications Engineering Team

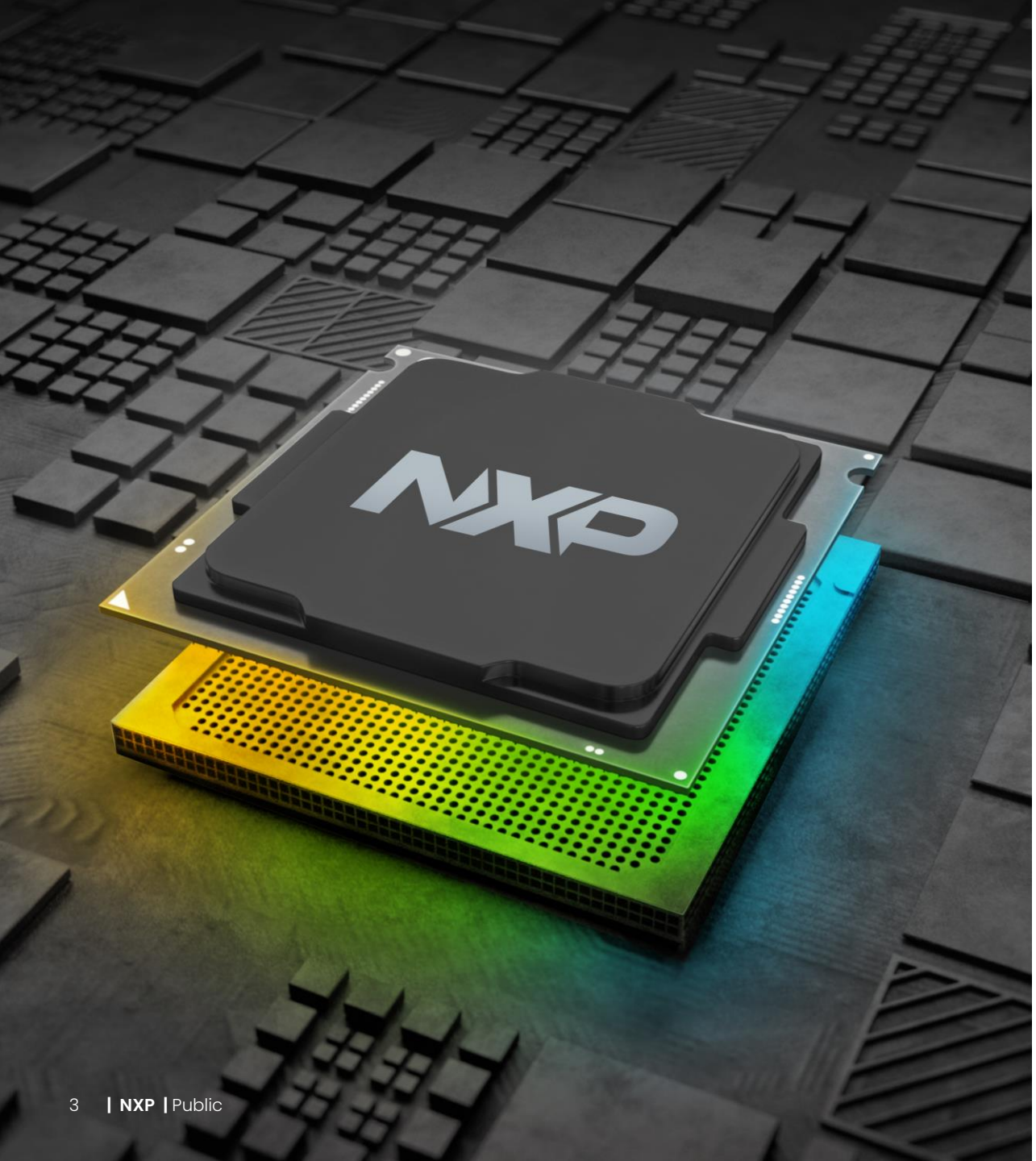
GPIS-BL, Automotive Processing

Feb 2024

| Public | NXP, and the NXP logo are trademarks of NXP B.V. All other product or service names are the property of their respective owners. © 2024 NXP B.V.

Glossary

- BCTU Body Cross Triggering Unit
- CH Channel
- CLK Clock
- CP Clock Prescaler
- DAOC Double Action Output Compare
- eDMA Enhanced Direct Memory Access
- eMIOS Enhanced Modular IO Subsystem
- GCP Global Clock Prescaler
- GFLAG Global Flag
- GPIO General Purpose Input and Output
- GPT General Purpose Timer
- GTBE Global Timebase Enable
- ICU Input Capture Unit
- IIB Internal Interface Bus
- IPM Input Period Measurement
- IPWM Input Pulse Width Measurement
- IRQ Interrupt Request
- ISR Interrupt Service Routine
- MC Modulus Counter
- MCAL Microcontroller Abstraction Layer
- MCB Modulus Counter Buffered
- MCL Microcontroller Library
- LLD Low Level Driver
- OCU Output Compare Unit
- OPWFMB Output Pulse Width Frequency Modulation Buffered
- OPWMB Output PWM Buffered
- OPWMT Output PWM with Trigger
- PEC Pulse/Edge Counting
- PWM Pulse Width Modulation
- RTD Real Time Driver
- SAIC Single Action Input Capture
- SAOC Single Action Output Capture
- TRGMUX Trigger Multiplexor
- UC Unified Channel



Agenda

1. Enhanced Modular IO Subsystem (eMIOS) Overview
2. eMIOS Modes
3. eMIOS Configuration Using RTD 3.0.0 (LLD)
4. Import and Run S32DS RTD 3.0.0 Examples
5. S32DS RTD 3.0.0 eMIOS Examples (LLD)
6. S32DS RTD 3.0.0 eMIOS Examples (MCAL)

01

Enhanced Modular IO Subsystem (eMIOS) Overview



Introduction

Enhanced Modular IO Subsystem (eMIOS) Overview

- What? – eMIOS provides user configurable Unified Channels (UC) that can generate and measure a range of timed events.
- The S32K3 family has up to three eMIOS instances, each featuring 24 UCs, two global counter buses and three dedicated counter buses.
- eMIOS can trigger eDMA channels and interconnects with TRGMUX.

Instance	S32K314/S32K324/S32K344/S32K358/ S32K348/S32K338/S32K328/S32K388	S32K311/S32K312/S32K322/ S32K342/S32K341
eMIOS_0	Yes	Yes
eMIOS_1	Yes	Yes
eMIOS_2	Yes	No

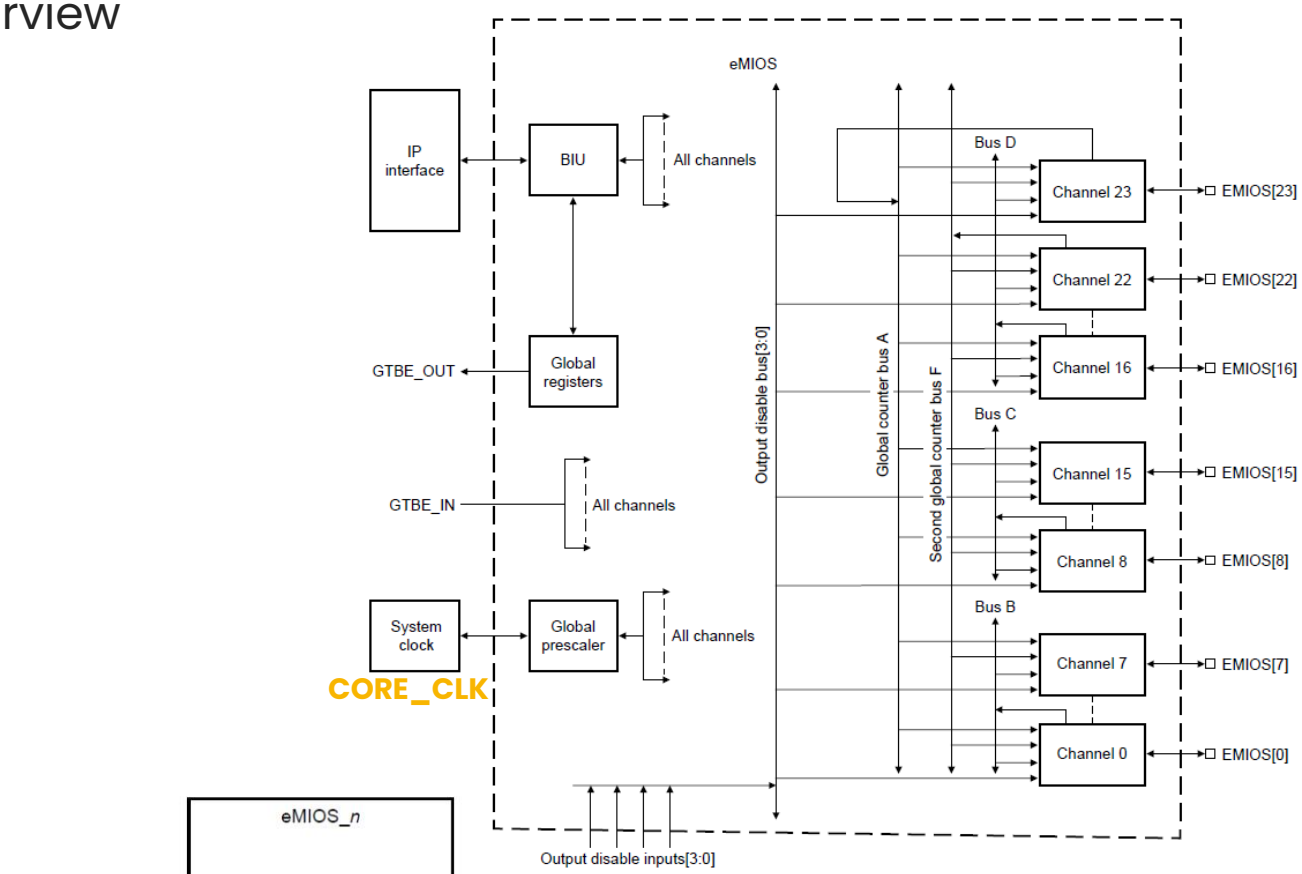
Parameter	eMIOS_0	eMIOS_1	eMIOS_2
Timer width	16 bits[1]	16 bits[1]	16 bits[1]
Number of channels	24	24	24
Channels configuration	See EMIOS channel configuration in 'Channel Types' slide		
Local Channel prescaler width	4	4	4
Number of global counter buses	2	2	2
Number of global prescaler	1	1	1
Global channel prescaler width	8	8	8

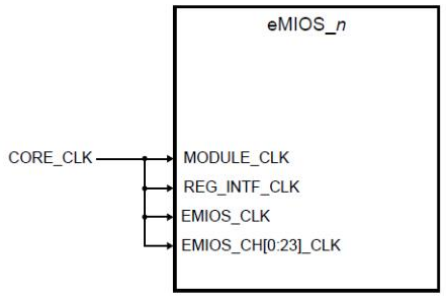
1. For S32K388: 24 bits

Introduction (Continued)

Enhanced Modular IO Subsystem (eMIOS) Overview

- Each eMIOS channel, except for the main counter bus channel (ch[23]), can trigger the BCTU.
- If GTBE_OUT is connected to the GTBE_IN input of one or more eMIOS instances, asserting it turns on the internal counters of those eMIOS instances simultaneously.
- eMIOS can disable its output using it's disable inputs[1].
- The eMIOS is clocked by CORE_CLK.
  - External clock is available only in MC and MCB modes.

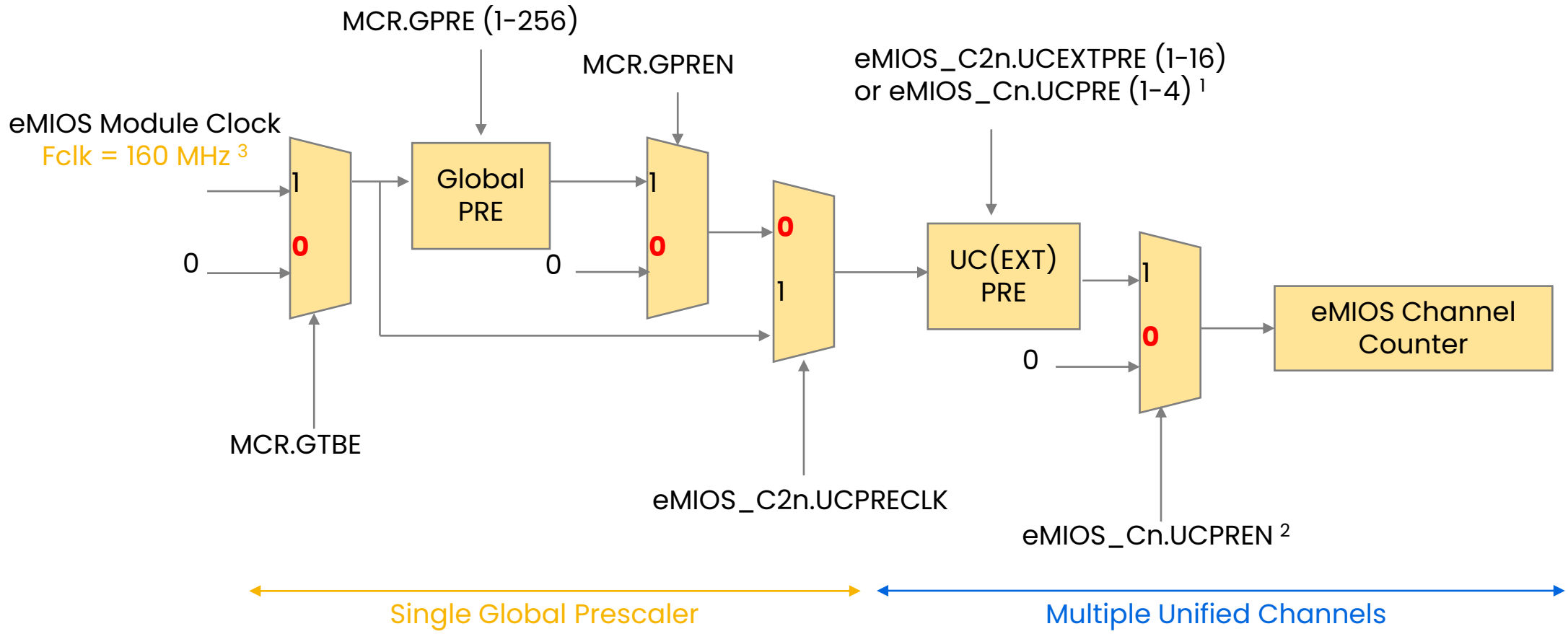




[1] Disable inputs[3:0] are driven from the flag out bits[11:8]. More details are included in the TRGMUX connectivity file attached to the Reference Manual.

Clock Structure

Enhanced Modular IO Subsystem (eMIOS) Overview

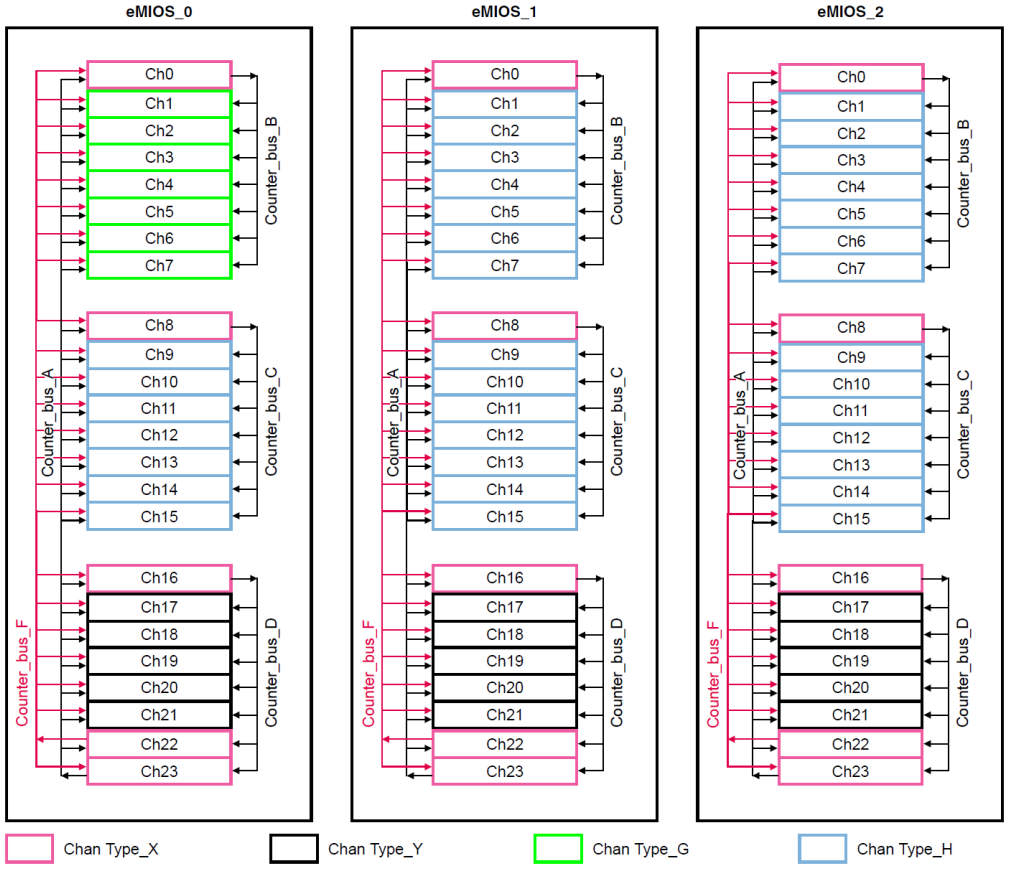


[1] UC (EXT) PRE has valid range of 1-16. If width>2 bits is required, eMIOS_C2n register is used. In this case the last 2 bits of eMIOS_C2n.UCEXTPRE[1:0] are mirrored in eMIOS_Cn.UCPRE[1:0].
[2] UCPREN = 1 is necessary to move internal counters.
[3] Single clock frequency is used throughout the design.

Channel Types

Enhanced Modular IO Subsystem (eMIOS) Overview

- eMIOS instances have four channel types each.
- Channel types X and G have internal counters.



Mode Description	Name	Ch TypeX	Ch TypeY	Ch TypeG	Ch TypeH
General Purpose Input / Output	GPIO	X	X	X	X
Single Action Input Capture	SAIC	X	X	X	X
Single Action Output Compare	SAOC	X	X	X	X
Modulus Counter	MC	X			
Modulus Counter Buffered (Up / Down)	MCB	X		X	
Input Pulse Width Measurement	IPWM			X	X
Input Period Measurement	IPM			X	X
Double Action Output Compare	DAOC			X	X
Output Pulse Width and Frequency Modulation Buffered	OPWFMB	X		X	
Center aligned Output PWM Buffered with dead time	OPWMCB			X	
Output Pulse Width Modulation Buffered	OPWMB	X	X	X	X
Output Pulse Width Modulation Trigger	OPWMT	X	X	X	X
Pulse Edge Counting	PEC			X	

Features Introduction

Enhanced Modular IO Subsystem (eMIOS) Overview

- The key features of the eMIOS modules are:
  - 24 UCs distributed across local counter buses.
  - Global counter bus A driven by UC23.
  - Global counter bus F driven by UC22.
  - Synchronized time bases shared throughout counter buses.
  - Dedicated time base for each channel, distinct from the counter buses.
  - Global Clock Prescaler (GCP).
  - One Clock Prescaler (CP) per channel.
  - Dedicated set of control and status registers for each UC.
  - 24-bit wide data registers.
  - Shadow flag register (GFLAG) to access all channel flags with one read access.
  - Ability to freeze the UC state for debug purposes.
  - Motor control capability.

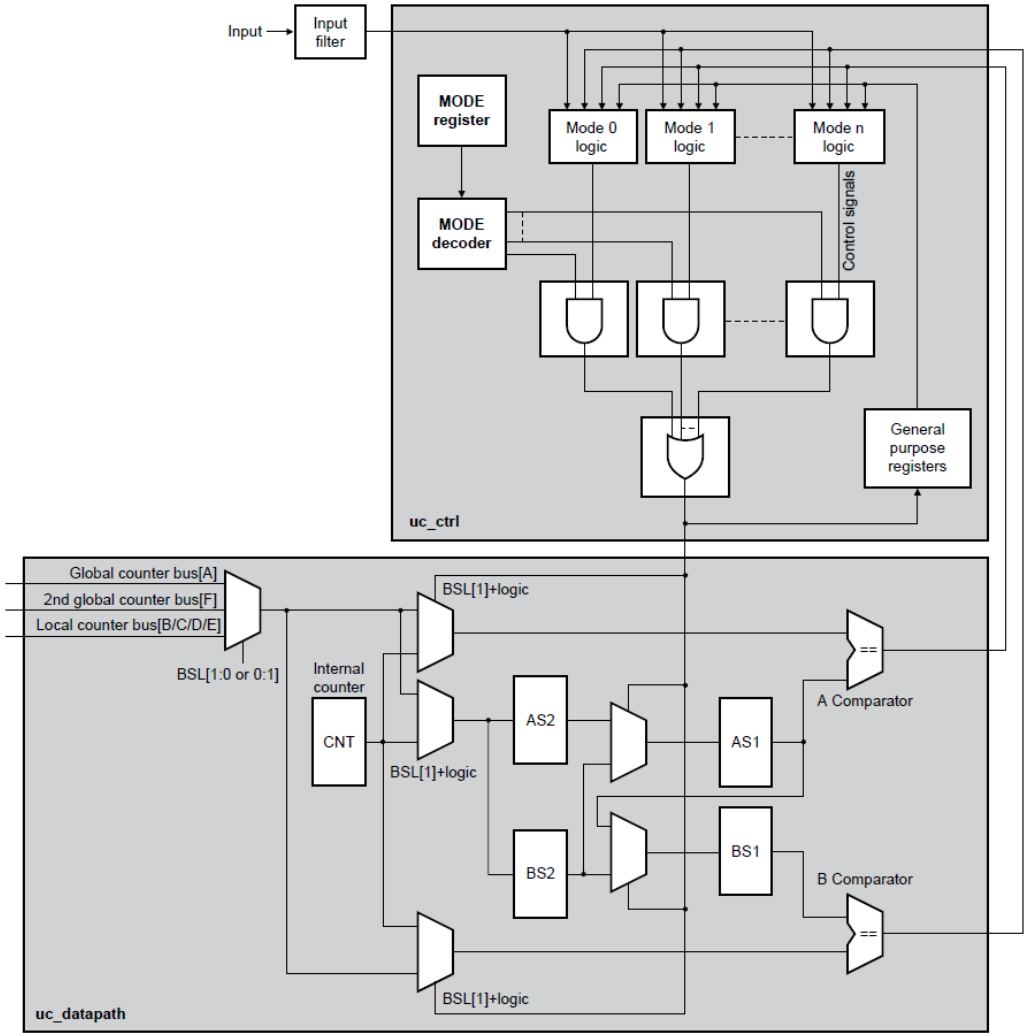
Base Address	eMIOS Registers
0	Module Configuration (MCR)
4	Global Flag (GFLAG)
8	Output Update Disable (OUDIS)
C	Disable Channel (UCDIS)
20-220	**UC CH0~23 Registers**

Base Address	UC CH0~23 Registers
0	UC A n (A0-A23)
4	UC B n (B0-B23)
8	UC Counter n (CNT0-CNT23)
C	UC Control n (C0-C23)
10	UC Status n (S0-S23)
14	Alternate Address n (ALTA0- ALTA23)
18	UC Control 2 n (C2_0-C2_23)

Features Introduction (Continued)

Enhanced Modular IO Subsystem (eMIOS) Overview

- The key features of the UCs are:
  - Two double-buffered data registers, An and Bn.
  - Two comparators, A and B.
  - An internal counter (CNTn[C]) that runs in all modes except GPIO.
  - An output flip-flop that holds the logic level to be applied to the output pin.
  - A status register, UC Status n (S0 - S23), that flags input capture and match events, indicates flag overruns and overflows, and shows the input and output pin states.
  - A control register, UC Control n (C0 - C23), that controls UC operation.



Features Introduction (Continued)

Enhanced Modular IO Subsystem (eMIOS) Overview

• General-Purpose Input and Output	GPIO
• Single Action Input Capture	SAIC
• Single Action Output Capture	SAOC
• Input Pulse Width Measurement	IPWM
• Input Period Measurement	IPM
• Double Action Output Compare	DAOC
• Pulse Edge Counting	PEC
• Modulus Counter	MC
• Modulus Counter Buffered	MCB
• Output Pulse Width and Frequency Modulation Buffered	OPWFMB
• Center Aligned Output PWM with Dead Time Insertion Buffered	OPWMCB
• Output PWM Buffered	OPWMB
• Output PWM with Trigger	OPWMT

Cn[MODE]

When changing MODE[6:0], the application software must first enter GPIO mode to reset the Unified Channels' internal functions properly.

Features Introduction (Continued)

Enhanced Modular IO Subsystem (eMIOS) Overview

- UC Control n (C0-C23):

  - Edge Polarity (EDPOL) settings:

For these modes:	EDPOL does this:	and EDPOL values mean:
Input	Selects which edge triggers the internal counter, an input capture, or the input capture flag. If EDPOL does not appear in the UC mode description, it has no effect on UC operation.	0b - Trigger on a falling edge 1b - Trigger on a rising edge
Output	Selects the logic level on the output pin.	0b - A match on comparator A deasserts the output flip-flop, while a match on comparator B sets it 1b - A match on comparator A asserts the output flip-flop, while a match on comparator B clears it

  - Edge Select (EDSEL) settings:

For these modes:	EDSEL does this:	and EDSEL values mean:
Input	Selects whether the internal counter is triggered by both edges of a pulse or by a single edge as defined by EDPOL. If EDSEL does not appear in the UC mode description, it has no effect on UC operation.	0b - Single-edge triggering on edge specified in EDPOL 1b - Both-edges triggering
GPIO in	Selects whether an edge asserts the input capture flag (S*n*[FLAG]).	0b - Sets S*n*[FLAG] on the edge specified in EDPOL 1b - Does not set S*n*[FLAG]
SAOC	Selects the behavior of the output flip-flop at each match.	0b - Drive the EDPOL value on the output flip-flop 1b - Toggle output flip-flop

  - MODE field values settings:

UC mode	MODE (binary)
GPIO (input)	000_0000
GPIO (output)	000_0001
SAIC	000_0010
SAIC with edge capturing	100_0010
SAOC	000_0011
IPWM	000_0100
IPM	000_0101
DAOC (with FLAG = 1 on B match)	000_0110
DAOC (with FLAG = 1 on A and B match)	000_0111
PEC (continuous)	000_1010
PEC (single shot)	000_1011
Reserved	000_1111
MC (up counter with clear on match start)	001_000p[1]
MC (up counter with clear on match end)	001_001p[1]
MC (up/down counter)	001_01pp[1]
Reserved	010_0100–010_0101
OPWMT	010_0110
Reserved	010_0111–100_1111
MCB (up counter)	101_000p[1]
Reserved	101_001p
MCB (up or down counter)	101_01pp[1]
OPWFMB	101_10p0[1]
Reserved	101_10p1
OPWMCB (with trailing edge dead time)	101_11p0[1]
OPWMCB (with leading edge dead time)	101_11p1[1]
OPWMB	110_00p0[1]
Reserved	110_0001–111_1111

Interrupts and eDMA

Enhanced Modular IO Subsystem (eMIOS) Overview

- Interrupt Source:
  - eMIOSn CH20 ~ CH23 -> eMIOSn ISR0
  - eMIOSn CH16 ~ CH19 -> eMIOSn ISR1
  - eMIOSn CH12 ~ CH15 -> eMIOSn ISR2
  - eMIOSn CH8 ~ CH11 -> eMIOSn ISR3
  - eMIOSn CH4 ~ CH7 -> eMIOSn ISR4
  - eMIOSn CH0 ~ CH3 -> eMIOSn ISR5 **n = 0, 1, 2**
- DMA Request:
  - eMIOSn CH0 -> DMAMUX0 -> eDMA CH0 ~ CH15
  - eMIOSn CH1 -> DMAMUX0 -> eDMA CH0 ~ CH15
  - eMIOSn CH9 -> DMAMUX0 -> eDMA CH0 ~ CH15
  - eMIOSn CH10 -> DMAMUX0 -> eDMA CH0 ~ CH15
  - eMIOSn CH16 -> DMAMUX1 -> eDMA CH16 ~ CH31
  - eMIOSn CH17 -> DMAMUX1 -> eDMA CH16 ~ CH31
  - eMIOSn CH18 -> DMAMUX1 -> eDMA CH16 ~ CH31
  - eMIOSn CH19 -> DMAMUX1 -> eDMA CH16 ~ CH31 **n = 0, 1, 2**

Channel Assignment Recommendations

Enhanced Modular IO Subsystem (eMIOS) Overview

- Channel Features Comparison:
  - Type G (7 channels) is full-featured. It has an internal timer, but it cannot be used by other channels.
  - Type X (15 channels) has OPWFMB. It has an internal timer and its MC/MCB timer bus can be used by other channels and no IPWM.
  - Type H (35 channels) has IPWM/IPM. It has no internal timer and no OPWFMB mode.
  - Type Y (15 channels) has only SAIC, OPWMB (frequency not adjustable), but no internal timer.
- PWM Input Channels:
  - Type H is suitable for PWM measurement – it requires a Type X channel running as counter bus (MC mode).
  - Type G is suitable for PWM measurement – it has full features.
- PWM Output Channels:
  - Type X is suitable for PWM output – both frequency and period are adjustable in OPWFMB mode.
  - Type G is suitable for PWM output – it has full features.
- One of the following channels for each eMIOS instance can be reserved for counter bus generation:
  - Type X Ch22 = counter bus F
  - Type X Ch23 = counter bus A

02

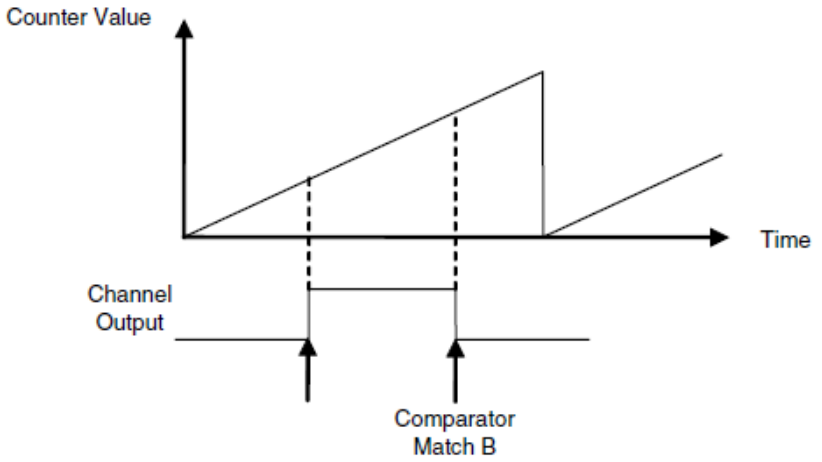
eMIOS Modes



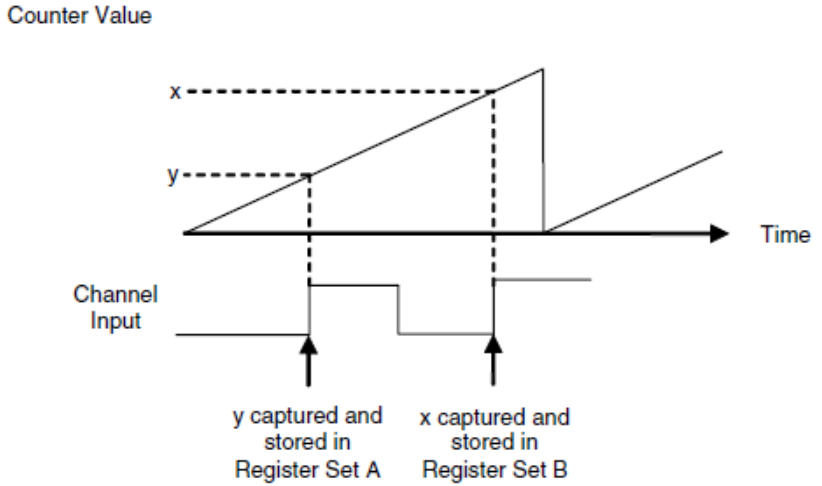
Modes' Principles of operation

eMIOS Modes

- The generation of timed outputs is based on hardware comparisons between values stored in the A and B registers and a time reference.
  - The values are continually compared to the selected counter. When a match occurs, this can correspond to a change of state of an output signal.
- For time measurements on input signals the UC captures and stores the value of the counter when the change of state occurs at the input.
  - Counter values are captured in the A and B registers.
- The hardware can also increment its counter as input edges are detected, allowing for support of edge counting modes.



eMIOS Principle of Operation: Output Modes



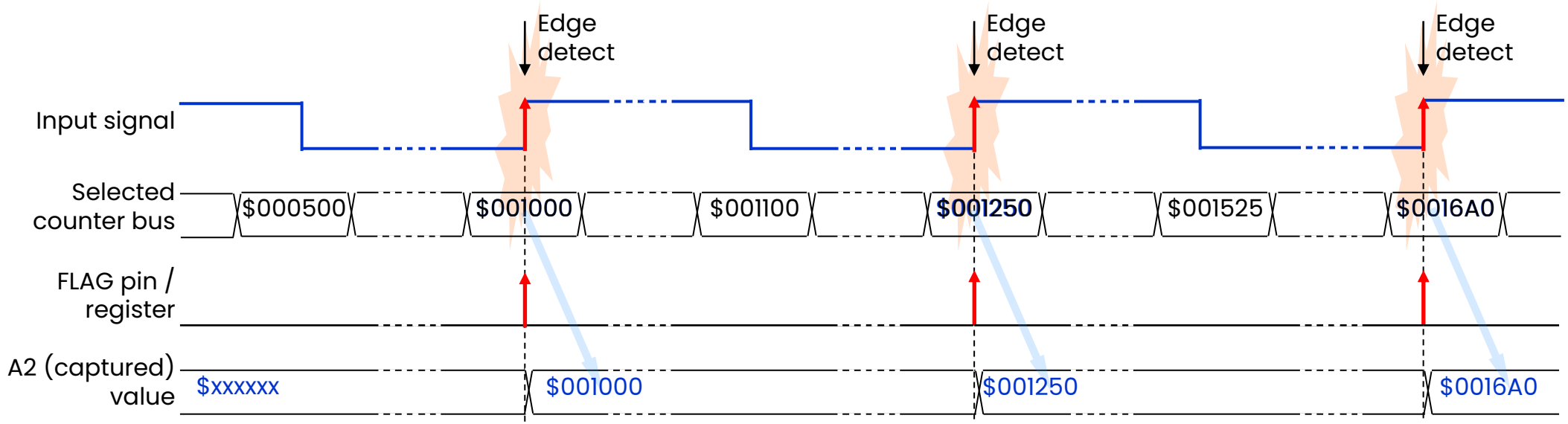
eMIOS Principle of Operation: Input Modes

Single Action Input Capture (SAIC)

eMIOS Modes

Type: X G Y H

- Returns the value of the counter bus on an edge match of an input signal.
  - Can use Internal or Modulus counter.
  - Can match on Rising, Falling or Toggle, determined by state of EDPOL, EDSEL.



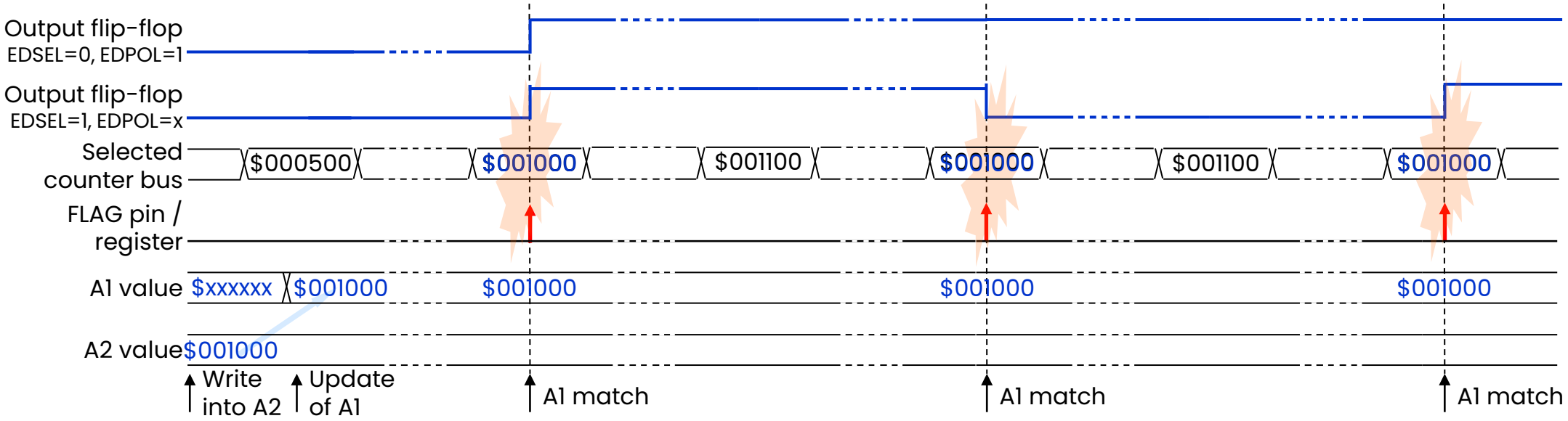
Notes:
- When an edge is detected, the flag is set, and counter bus value is captured in register A2. User reads this value from UCA[n] register.
- UCB[n] = Cleared and cannot be written.

Single Action Output Capture (SAOC)

eMIOS Modes

Type: X G Y H

- Generates an output on a counter bus match.
  - Can use Internal or Modulus counter.
  - Can set output to go High, Low or Toggle, determined by state of EDPOL, EDSEL.



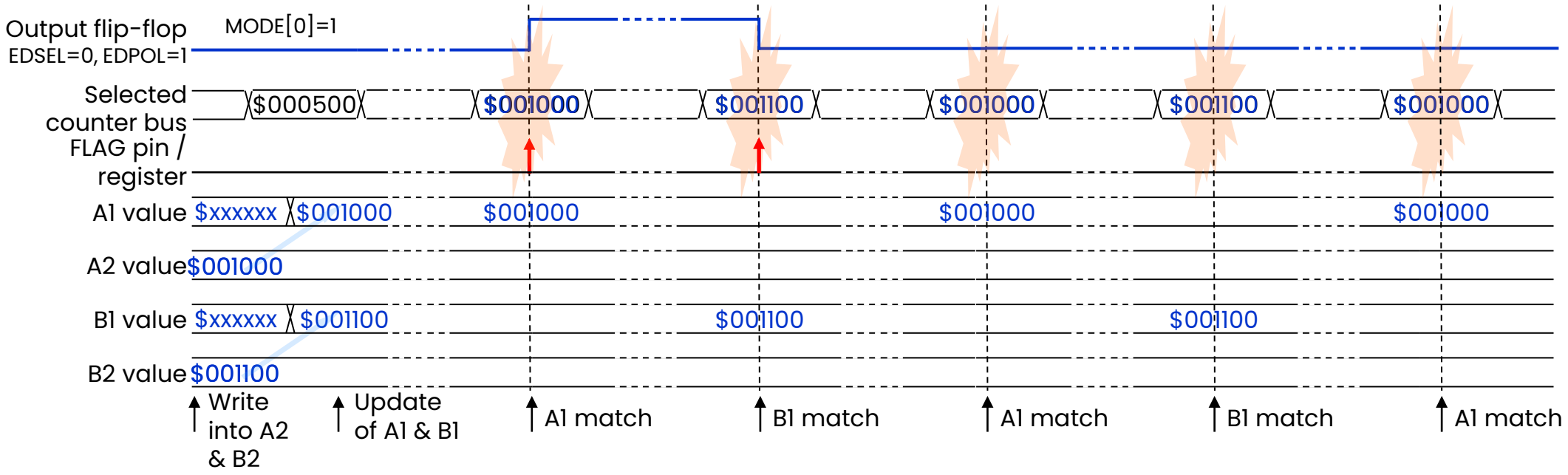
Notes:
- Write the desired counter bus value to create a match into UCA[n] (A2n) which is buffered into A1.
- A comparator match of A1 results in an output event, defined by status of EDPOL and EDSEL.

Double Action Output Compare (DAOC)

eMIOS Modes

Type: G H

- Generates an output pulse.
  - Can use Internal or Modulus counter.
  - Can be configured to measure between 2 HIGH or 2 LOW edges (EDPOL bit).



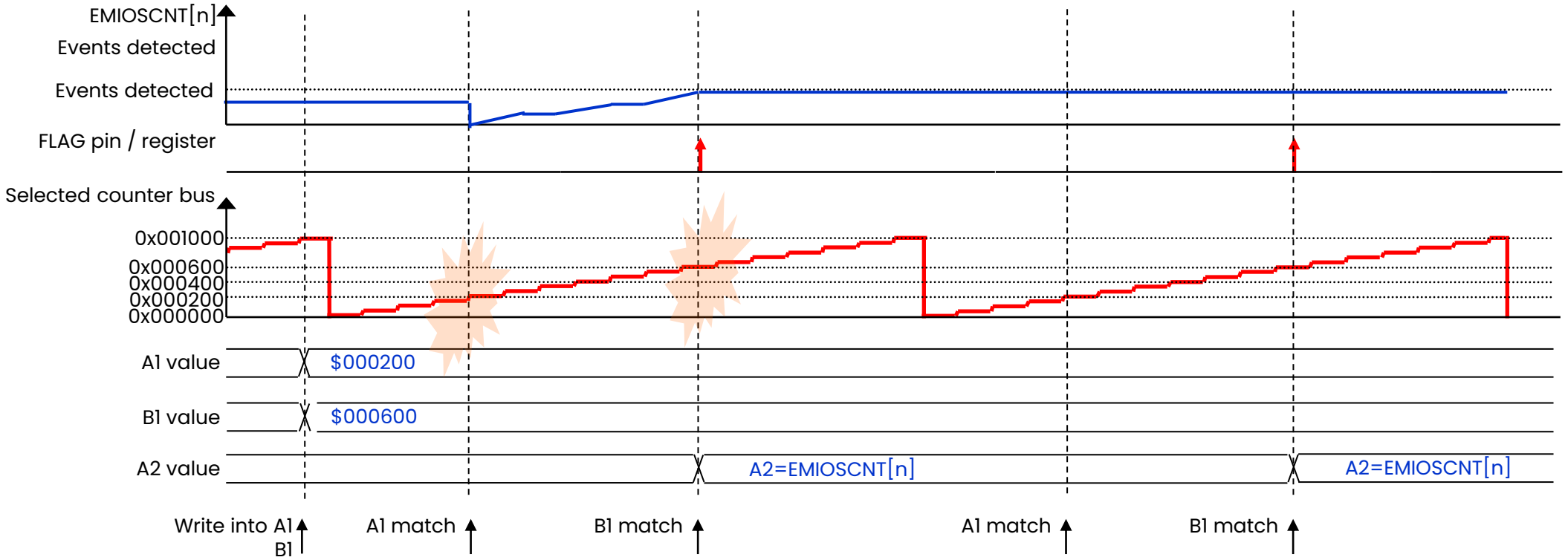
Notes:
- Write the desired pulse leading edge into UCA[n] (A2n) and the falling edge into UCB[n] (B2n), which are buffered into A1 and B1.
- On a comparator A match, the output is set to the value of EDPOL. FLAG is set if MODE[0]=1.
- On a comparator B match, the output is set to the inverse of EDPOL. FLAG is set.

Pulse/Edge Counting (PEC) – Single Shot

Type
G

eMIOS Modes

- Returns the number of pulses or edges detected on the input for a desired time window.



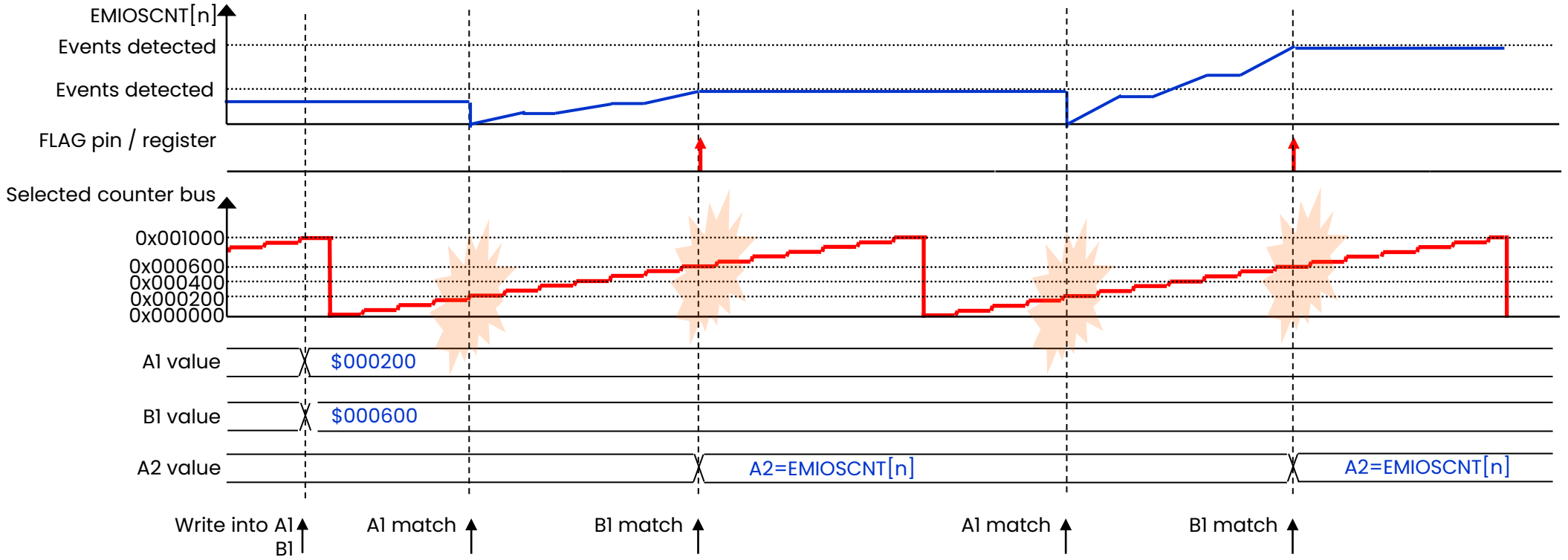
Notes:
- MODE[0] = 1 bit selects single shot operation.
- The polarity and the triggering edge is selected by EDSEL and EDPOL bits in EMIOSC[n] register.
- A1 holds the start time and B1 holds the stop time for the time window. A2 always holds the latest available measurement.
- The next match between comparator A and the selected time base has no effect, until a new write to register A is performed.

Pulse/Edge Counting (PEC) – Continuous Mode

Type
G

eMIOS Modes

- Returns the number of pulses or edges detected on the input for a desired time window.



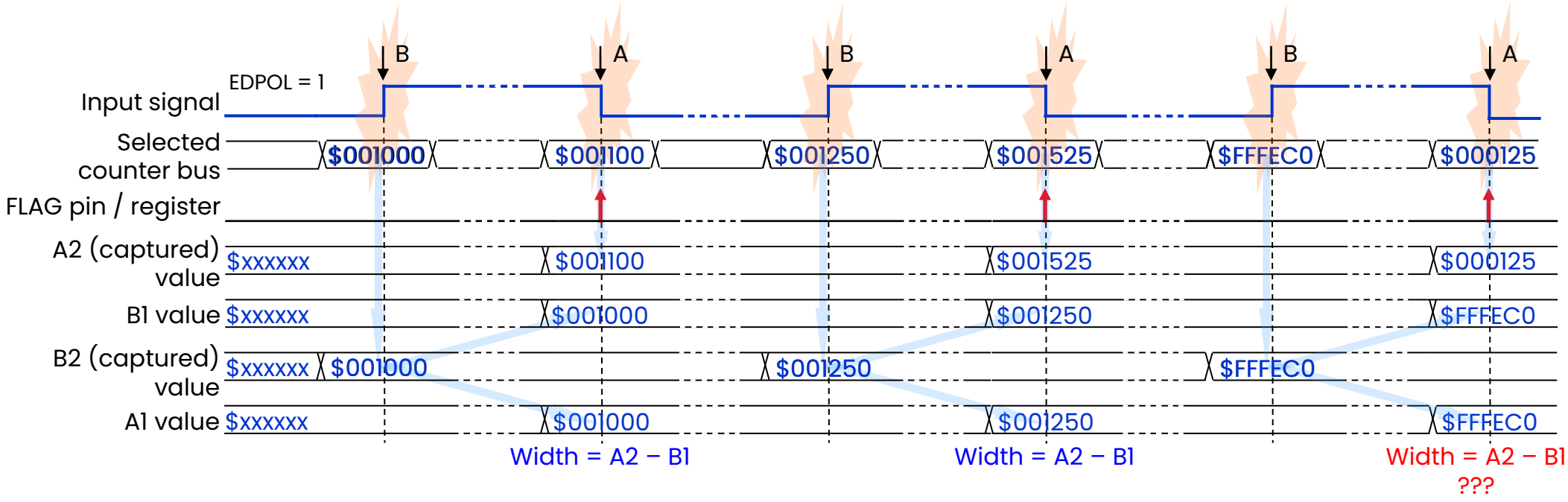
Notes:
- MODE[0] = 0 bit selects continuous operation.
- The polarity and the triggering edge is selected by EDSEL and EDPOL bits in EMIOSC[n] register.
- A1 holds the start time and B1 holds the stop time for the time window.
- Register A2 always holds the latest available measurement, providing coherent data at any time after the first FLAG had occurred.

Input Pulse Width Measurement (IPWM)

eMIOS Modes

Type: G H

- Determines the width of an input pulse (in counter bus clock ticks).
  - Can use Internal or Modulus counter.
  - Can be configured to measure HIGH or LOW pulses by state of EDPOL bit (EDPOL=1 for HIGH).



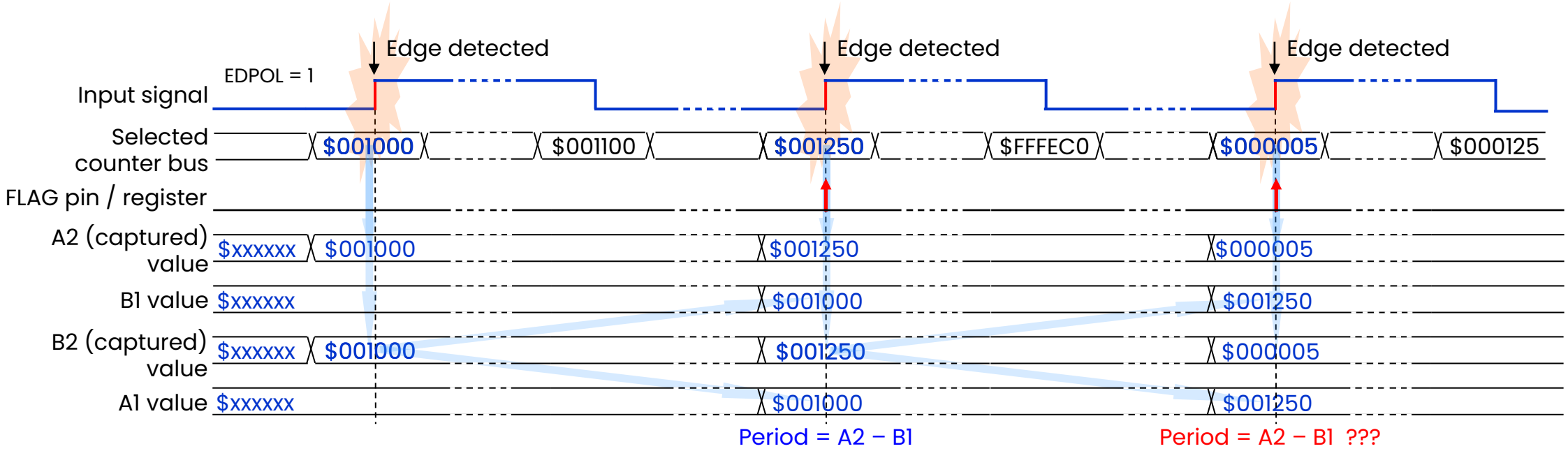
Notes:
- Leading edge is captured into B2[n] (EDPOL determines if leading edge is high or low).
- Trailing edge is captured into A2[n] and Flag is set.
- Pulse width is calculated by subtracting UCBn (B1) from UCAn (A2).
- Caution: If pulse has spanned a counter bus period, then Width = (UCAn + Counter Bus Period) - UCBn

Input Period Measurement (IPM)

eMIOS Modes

Type: G H

- Determines the period of an input pulse (in counter bus clock ticks).
  - Can use Internal or Modulus counter.
  - Can be configured to measure between 2 HIGH or 2 LOW edges (EDPOL bit).



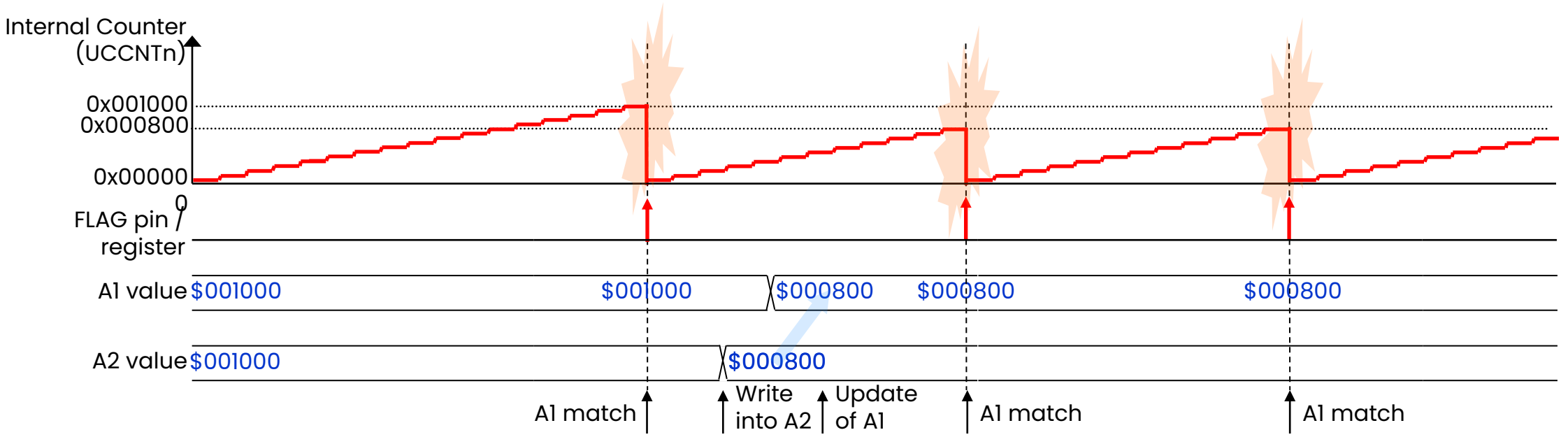
Notes:
- When the edge of the selected polarity is detected, counter value is captured into A2[n] and B2[n], the data previously held in B2[n] is captured into A1[n] and B1[n], and Flag is set.
- Period is calculated by subtracting UCBn (B1) from UCAn (A2).
- Caution: If input signal period has spanned a counter bus period, then Period = (UCAn + Counter Bus Period) - UCBn

Modulus Counter Mode (MC) – UP Counter

Type			
X			

eMIOS Modes

- Generates a time base which can be shared with other channels through the internal counter bases.
  - Can use Internal or External (input channel pin) counter.



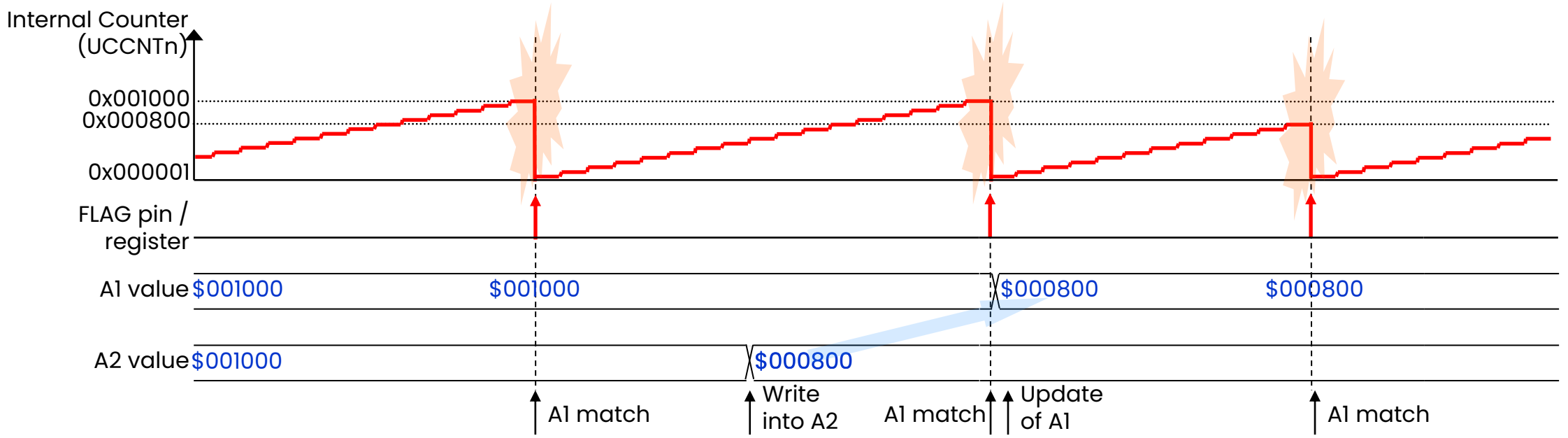
Notes:
- On a comparator A match, FLAG is set, and the internal counter is set to value $0.
- A change of the A2 register updates the A1 register at the next clock.
- Caution: If when entering MC mode, the internal counter value is greater than that of register UCA[n], then it will wrap at the maximum counter value ($FFFFFF) before matching A1.

Modulus Counter Buffer Mode (MCB) – UP Counter

Type			
X	G		

eMIOS Modes

- Generates a time base which can be shared with other channels through the internal counter bases.
  - Can use Internal or External (input channel pin) counter.



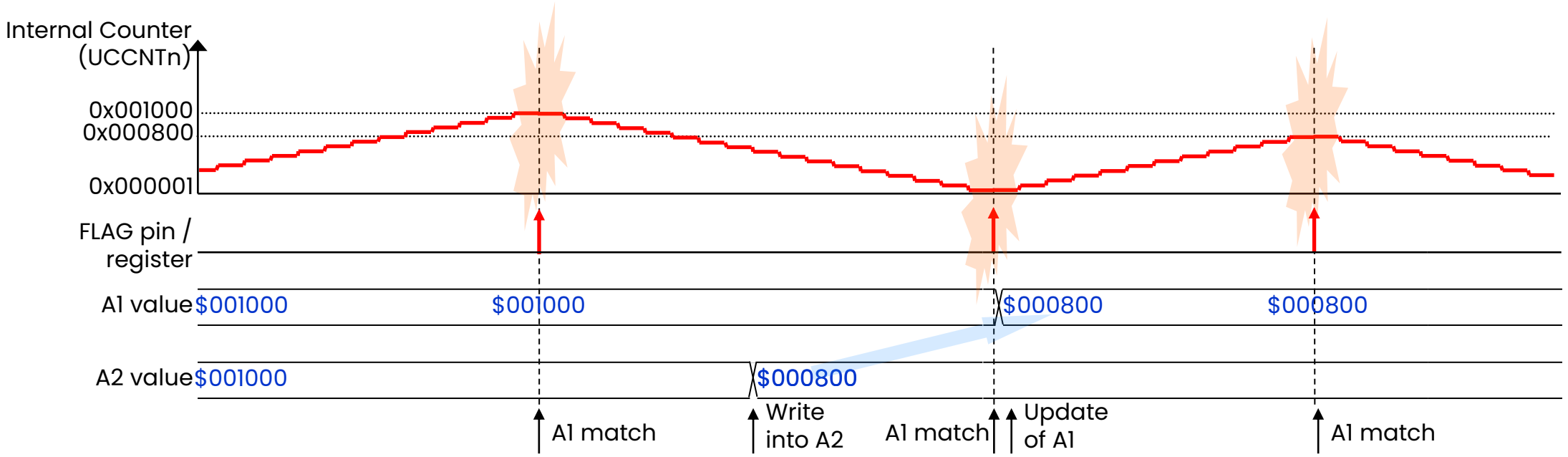
Notes:
- On a comparator A match, FLAG is set, and the internal counter is set to value $1.
- Allows for smooth transitions; a change of A2 register updates A1 register when the internal counter reaches the value $1.
- Caution: If when entering MCB mode the internal counter value is greater than that of register UCA[n], then it will wrap at the maximum counter value ($FFFFFF) before matching A1.

Modulus Counter Buffer Mode (MCB) – UP/DOWN Counter

Type			
X	G		

eMIOS Modes

- Generates a time base which can be shared with other channels through the internal counter bases.
  - Can use Internal or External (input channel pin) counter.



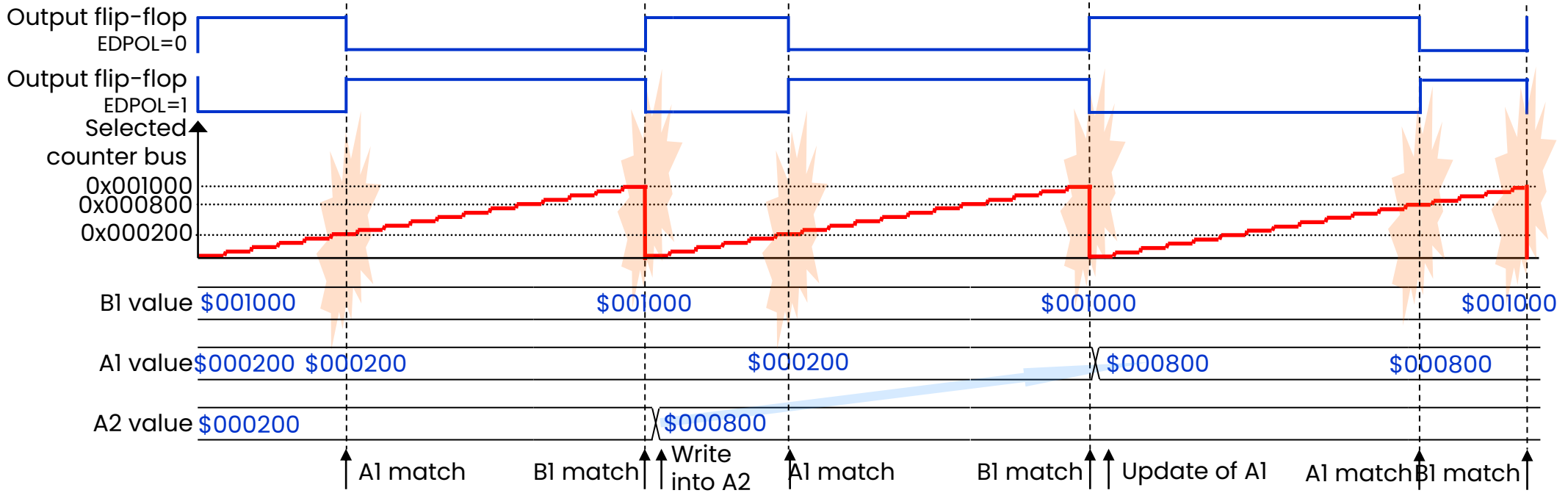
Notes:
- On a comparator A match, FLAG is set, and the internal counter is set to value $1.
- Allows for smooth transitions; a change of A2 register updates A1 register when the internal counter reaches the value $1.
- Caution: If when entering MCB mode the internal counter value is greater than that of register UCA[n], then it will wrap at the maximum counter value ($FFFFFF) before matching A1.

Output Pulse Width and Frequency Modulation buffered (OPWFMB)

eMIOS Modes

Type: X G

- Generates a simple output PWM signal.
  - Requires internal counter.
  - EDPOL allows selection between active HIGH or active LOW duty cycle.



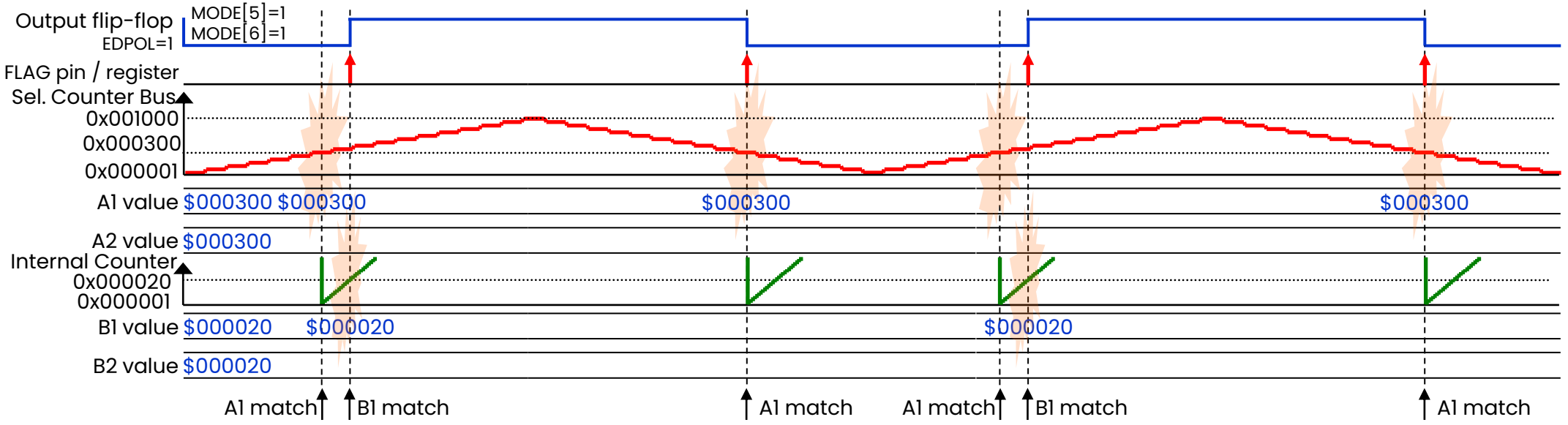
Notes:
- Duty Cycle = UCA[n] (A1) + 1, Period = UCB[n] (B1) + 1
- On comparator A1 match, output pin is set to value of EDPOL.
- On comparator B1 match, output pin is set to complement of EDPOL, and internal counter is reset.
- The transfers from register B2[n] to B1[n] and from register A2[n] to A1[n] are performed at the first clock of the next cycle.
- FLAGS can be generated only on B1 matches or on both A1 and B1 matches, depending on MODE[5] bit.

Center Aligned Output PWM with Dead Time Insertion Buffered (OPWMCB)

eMIOS Modes

Type
G

- Generates a centered aligned PWM output signal with dead time insertion.
  - MODE[6] bit selects between trailing and leading dead time insertion.
  - EDPOL allows selection between active HIGH or active LOW duty cycle. Requires MCB UP/DOWN.



Notes:

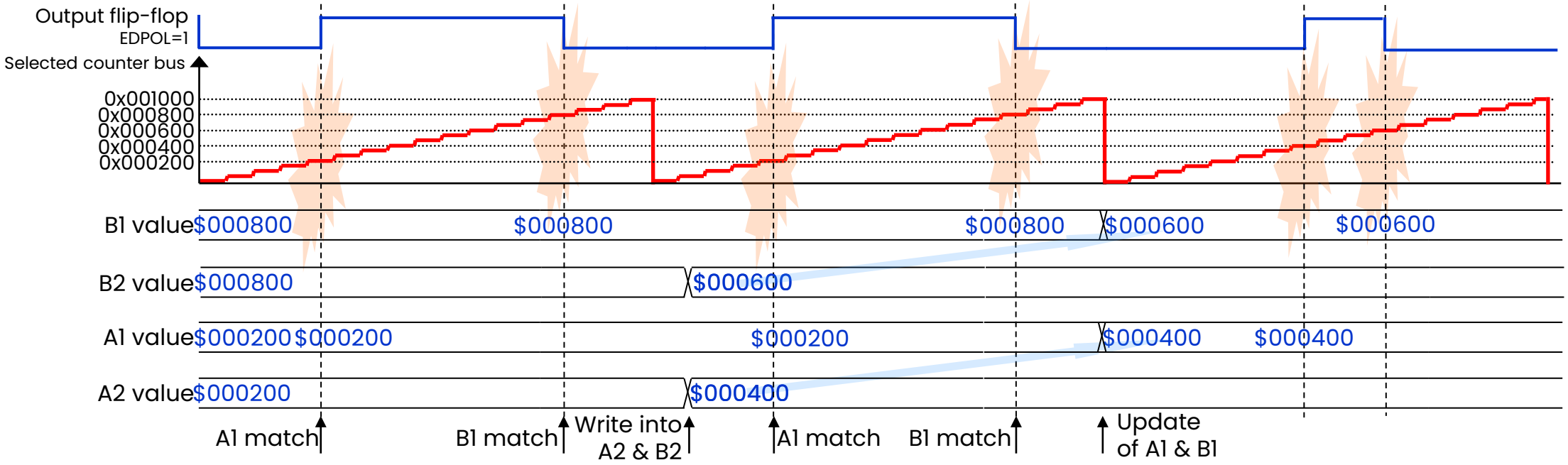
- On the selected dead time insertion edge:
  - On comparator A1 match (selected counter bus), the internal counter is set to $1.
  - On comparator B1 match (internal counter), output pin is set to the value of EDPOL.
- On the non-selected dead time insertion edge:
  - On comparator A1 match, output pin is set to complement of EDPOL and internal counter set to $1.
- Period = MCB Period, Dead Time = B1[n], Duty Cycle = 2 * (Period - A1[n]) – Dead Time
- FLAGS can be generated only on B1 matches or on both A1 and B1 matches, depending on MODE[5] bit.
- The transfers from register B2[n] to B1[n] and from register A2[n] to A1[n] are performed at the first clock of the next cycle.
- To achieve 100% duty cycle, A1[n] must be set to $1. To achieve 0% duty cycle, A1[n] must be set to a value greater than the maximum value of the selected time base.
- Caution: The internal counter should not reach $0 as consequence of a rollover.

Output pulse Width Modulation Buffered (OPWMB)

eMIOS Modes

Type: X G Y H

- Generates a simple output PWM signal.
  - Uses Modulus counter.
  - EDPOL allows selection between active HIGH or active LOW duty cycle.



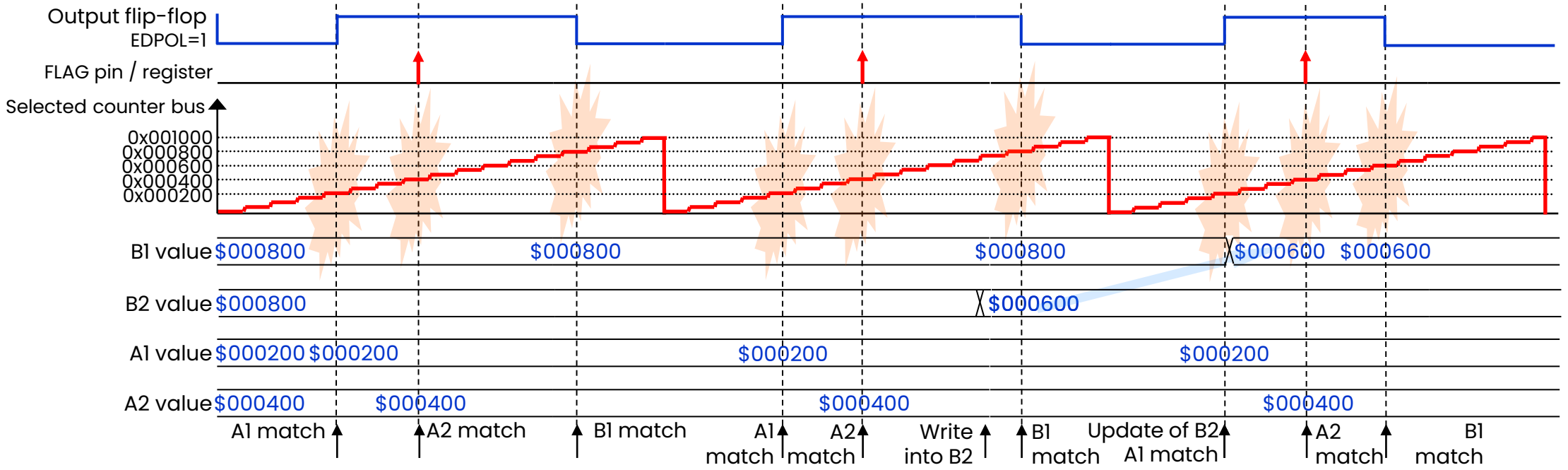
Notes:
- Write UCA[n] (A1) with leading edge. Write UCB[n] (B1) with trailing edge.
- On comparator A1 match, output pin is set to value of EDPOL.
- On comparator B1 match, output pin is set to complement of EDPOL.
- The transfers from register B2[n] to B1[n] and from register A2[n] to A1[n] are performed at the first clock of the next cycle.
- FLAGS can be generated only on B1 matches or on both A1 and B1 matches, depending on MODE[5] bit.

Output Pulse Width Modulation with Trigger (OPWMT)

eMIOS Modes

Type: X G Y H

- Generates a PWM signal with a fixed offset and a trigger signal.
  - Intended to be used with other channels in the same mode with shared common time base.
  - This mode is particularly useful in the generation of lighting PWM control signals.



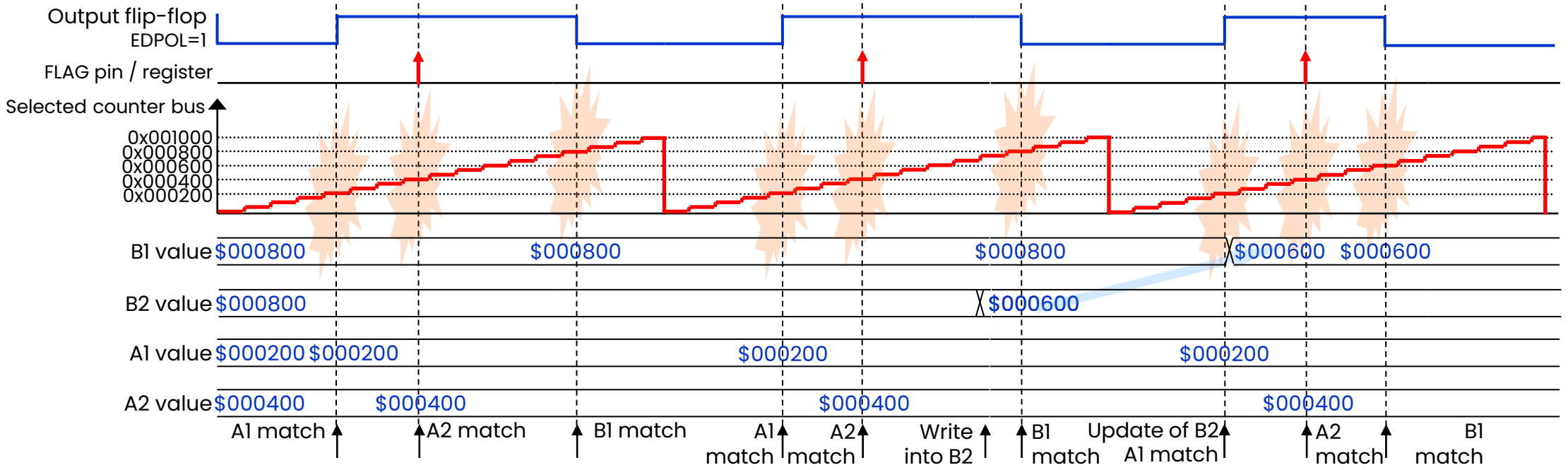
Notes:
- A1[n] defines the leading edge, B1[n] the trailing edge, A2[n] the generation of a FLAG event.
- On comparator A1 match, output pin is set to value of EDPOL.
- On comparator A2 match, FLAG is set (and can allow to synchronize with other events, i.e., AD conversion).
- On comparator B1 match, output pin is set to complement of EDPOL.
- The transfers from register B2[n] to B1[n] is performed at every match of register A1.

Output Pulse Width Modulation with Trigger (OPWMT) (Continued)

Type: X G Y H

eMIOS Modes

- Generates a PWM signal with a fixed offset and a trigger signal.
  - Intended to be used with other channels in the same mode with shared common time base.
  - This mode is particularly useful in the generation of lighting PWM control signals.



Notes:
- To achieve 0% duty cycle, both registers A1 and B1 must be set to the same value.
- To achieve 100% duty cycle, the register B1 must be set to a value greater than the maximum value of the selected time base.
- Caution: A2[n] value is not readable. A1[n] value is not writable and therefore cannot be configured in this mode (use GPIO mode).
- Caution: Registers loaded with $0 will not produce matches if the time base is driven by a channel in MCB mode.
- Caution: A2[n] is not buffered, its update is immediate.

03

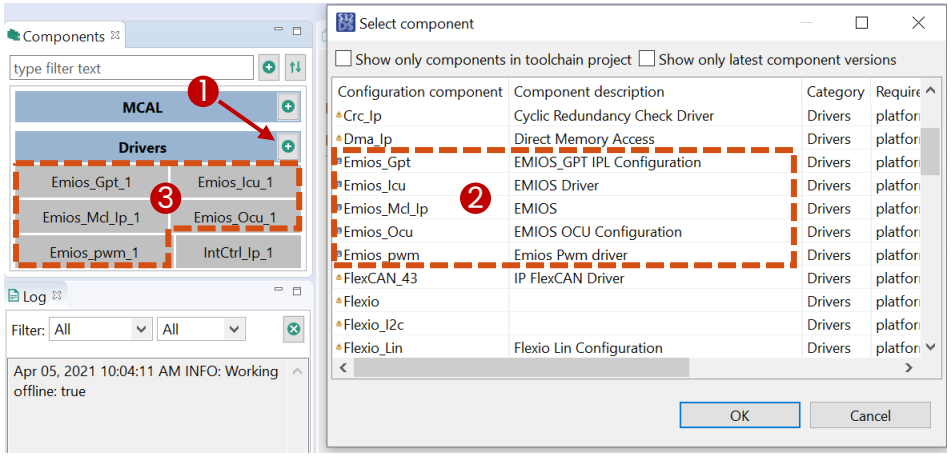
eMIOS Configuration Using RTD 3.0.0 (LLD)



Overview of eMIOS components of RTD

eMIOS Configuration Using RTD 3.0.0 (LLD)

- The ConfigTools provide five RTD eMIOS drivers to implement diverse applications:
  - eMIOS Microcontroller Library **Emios_Mcl_Ip**
  - eMIOS General Purpose Timer **Emios_Gpt**
  - eMIOS Output Compare Unit **Emios_Ocu**
  - eMIOS Pulse Width Modulation **Emios_Pwm**
  - eMIOS Input Capture Unit **Emios_Icu**



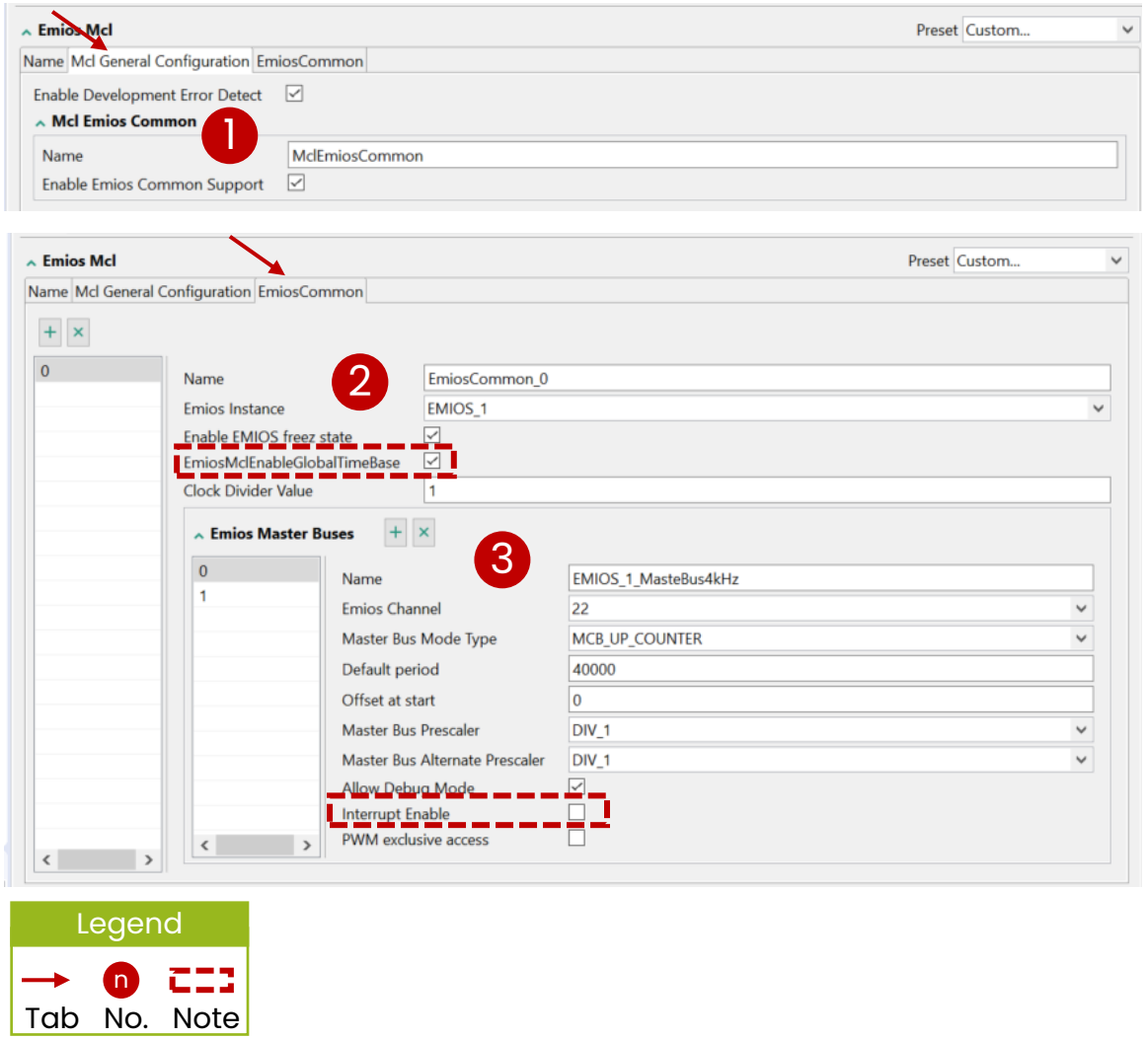
- Need to pay attention to:
  - **Emios_Mcl_Ip** must be added in your project if you want to use the other four drivers.
  - Each eMIOS RTD module has its own IRQ function, and you can set a callback function for your application. All the handler entries allocate in MCL IRQ files.
    - **EMIOS_IRQ → Emios_Pwm_Ip_IrqHandler(uint8 Instance, uint8 Channel) → Callback Function/Notification**
  - General process for eMIOS initialization:
    - **Initialize instance (MCL) → Initialize channel (Gpt, Ocu, Pwm, Icu) → Other (start, notification, etc.)**

Emios_Mcl_IP

eMIOS Configuration Using RTD 3.0.0 (LLD)

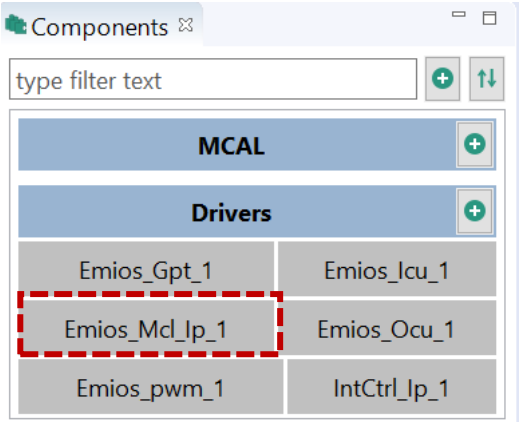
- **MCL – General process in ConfigTools**

1. Enable eMIOS Common Support.
   - If not, EmiosCommon is unavailable.
   - Enable Error Detect is needed.
2. Configure Global eMIOS.
   - Enable eMIOS instance freeze state and global time base.
   - Set global clock divider value: 1 ~ 256.
3. Configure eMIOS master buses.
   - Select HW channel.
   - Set master bus mode.
   - Set period and offset at start and channel prescaler.
   - Enable or disable debug mode and interrupts.
   - Master bus channel must not be configured to other mode.



Emios_Mcl_IP (Continued)

eMIOS Configuration Using RTD 3.0.0 (LLD)



- **Use tips:**
  - Function: initialize eMIOS instance.
  - Manage the other four eMIOS modules, including ISRs and supply master timer buses.
  - MC and MCB modes' support.
  - There is no MCL_EMIOS_ISR; it's not recommended to enable the interrupt.
  - Enable the last global time base if you want to start multiple instances simultaneously.
- **Key APIs[1]:**
  - Emios_Ip_CommonStatusType **Emios_Mcl_Ip_Init**(uint8 instance, const Emios_Mcl_Ip_ConfigType* const pConfig)
  - Emios_Ip_CommonStatusType **Emios_Mcl_Ip_SetCounterBusPeriod**(uint8 hwInstance, uint8 hwChannel, uint16 period)
  - void **Emios_Mcl_Ip_EnableChannel**(uint8 instance, uint8 u8HwChannel)
    - Close UC clock by setting UCDIS bit.
  - void **Emios_Mcl_Ip_ComparatorTransferEnable**(uint8 instance, uint8 u8HwChannel)
    - Disables transfers from AS2 to AS1 and BS2 to BS1 on a per-channel basis by setting OUDIS bit.
  - void **Emios_Mcl_Ip_SetReloadInterval**(uint8 hwInstance, uint8 hwChannel, uint8 interval)
    - Control load signal frequency by setting UCRELDEL bit in C2_n.
  - boolean **Emios_Mcl_Ip_ValidateChannel**(uint8 hwInstance, uint8 hwChannel)
    - Validate whether channel is initialized through global variable Emios_Ip_ChState.

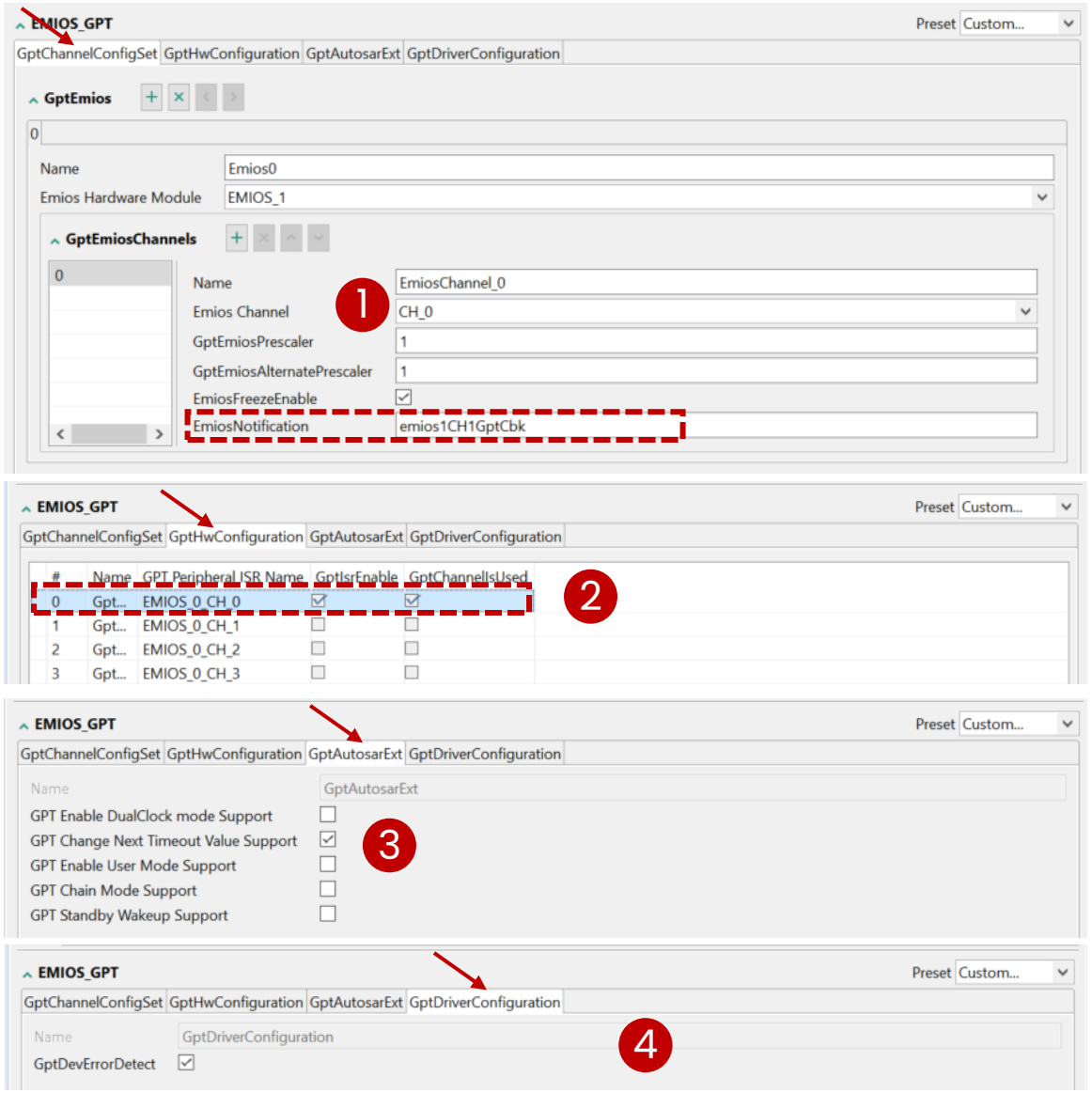
[1] Not all the API is listed here.

Emios_Gpt

eMIOS Configuration Using RTD 3.0.0 (LLD)

- **GPT – General process in ConfigTools**

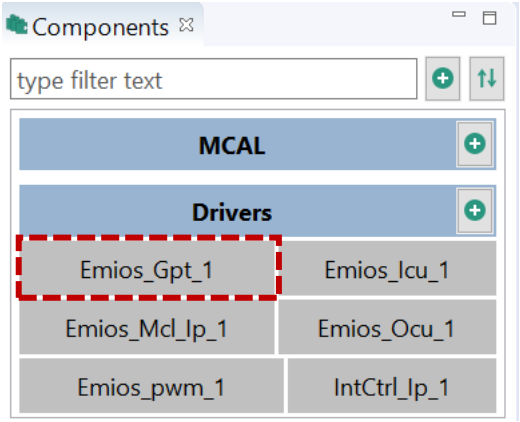
1. Configure GPT channel and local prescaler.
   - Only the channels which have counters can be selected.
   - Fill the notification blank with your defined callback function if needed.
2. Enable GPT channel and GPT ISR.
   - If not, GPT channel cannot work.
   - This enablement doesn't set FEN bit.
3. Enable AUTOSAR support if needed.
   - Chain mode doesn't support for eMIOS.
4. Enable error detect if needed.





Emios_Gpt (Continued)

eMIOS Configuration Using RTD 3.0.0 (LLD)



- **Use tips:**
  - Function: configure General Purpose Timer (GPT) using eMIOS channel.
    - The frequency is up to 160MHz with 16 bits width counter (for S32K388: 24 bits).
  - MCB and GPI modes' support.
  - Call **Emios_Gpt_lp_EnableChannelInterrupt**() after GPT channel initialization if needed.
  - The GPT starts up count after calling **Emios_Gpt_lp_StartTimer**().
  - Change period dynamically by calling **Emios_Gpt_lp_ChangeNextTimeoutValue**(), if enabled.
  - Though GPT runs at MCB mode, GPT is different from master bus at the level of RTD.
  - **Emios_Gpt_lp_IrqHandler**() is in Emios_Gpt_lp.c.
- **Key APIs[1]:**
  - void **Emios_Gpt_lp_InitChannel**(uint8 instance, const Emios_Gpt_lp_ChannelConfigType *configPtr)
  - void **Emios_Gpt_lp_StartTimer**(uint8 instance, uint8 channel, uint16 compareValue)
    - Start timer by entering MCB mode.
  - void **Emios_Gpt_lp_StopTimer**(uint8 instance, uint8 channel, uint16 compareValue)
    - Stop timer by going back to GPI mode.
  - void **Emios_Gpt_lp_EnableChannelInterrupt**(uint8 instance, uint8 channel)
  - void **Emios_Gpt_lp_ChangeNextTimeoutValue**(uint8 instance, uint8 channel, uint16 value)
    - Change period dynamically without stopping timer.

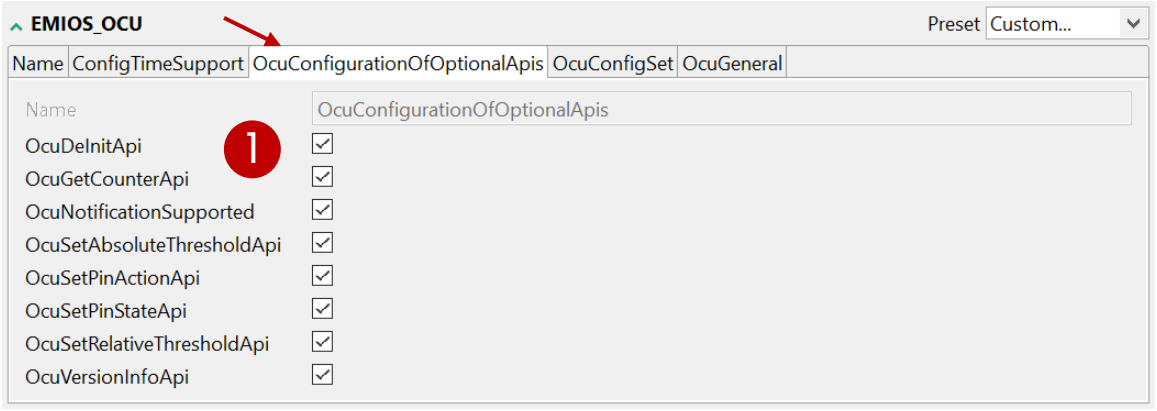
[1] Not all the API is listed here.

Emios_Ocu

eMIOS Configuration Using RTD 3.0.0 (LLD)

- **OCU – General process in ConfigTools**

1. Enable optional APIs if needed.
2. Configure OCU channel and default threshold and period.
   - The selected channel must be added in OcuHWSpecificSettings.
   - The default threshold should smaller than max counter value (period).
3. Configure pin default state and pin action.
   - Set high, set low, toggle or disable.
4. Set up HW channel, including clock source and prescaler.
   - Select a master bus working in MCB mode.
   - If not, OcuClockSource won't take effect.
5. Enable OCU ISR and channel used.
   - If not, GPT channel cannot work.
   - This enablement doesn't set FEN bit.





Emios_Ocu (Continued)

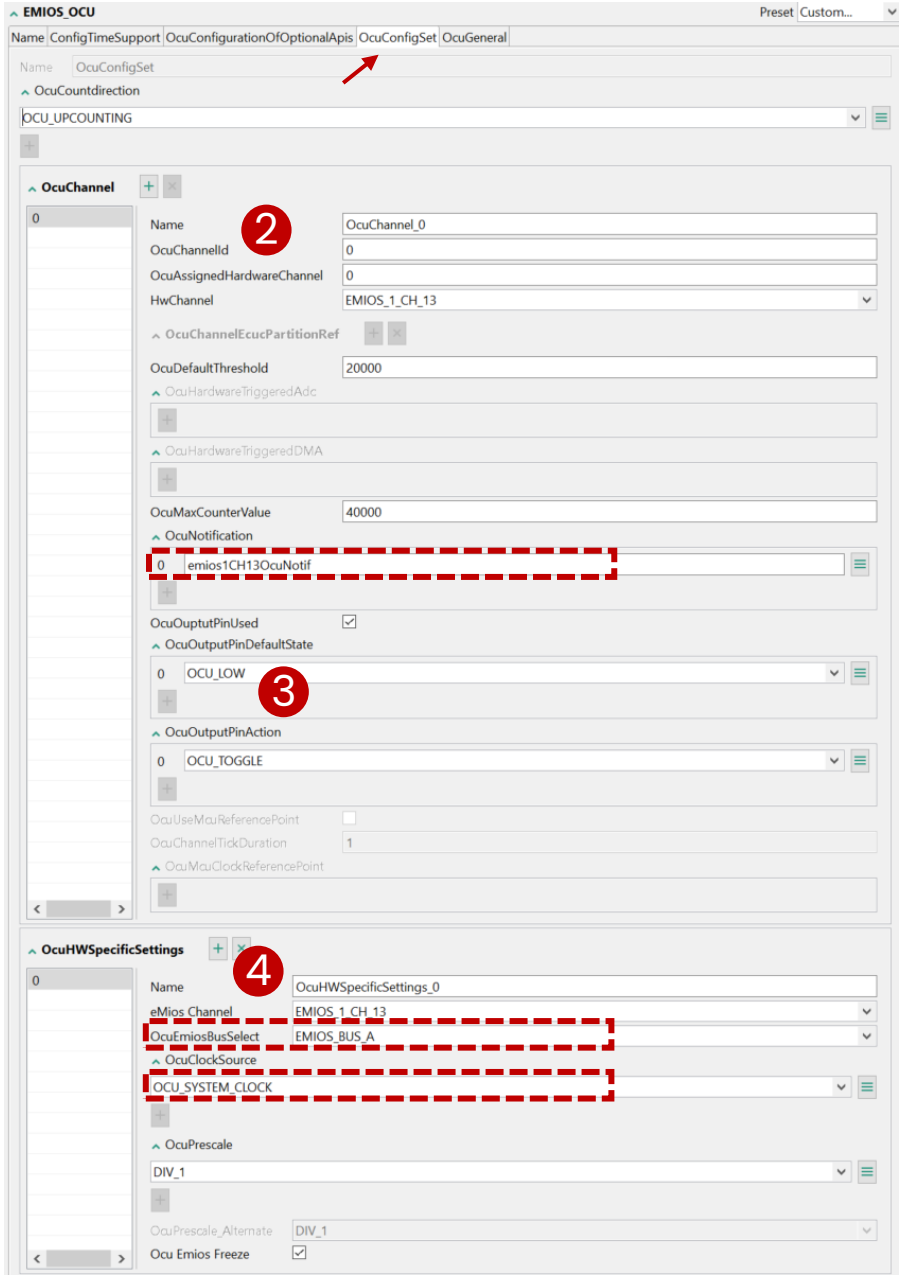
eMIOS Configuration Using RTD 3.0.0 (LLD)

- **OCU – General process in ConfigTools**

1. Enable optional APIs if needed.
2. Configure OCU channel and default threshold and period.
   - The selected channel must be added in OcuHWSpecificSettings.
   - The default threshold should smaller than max counter value (period).
3. Configure pin default state and pin action.
   - Set high, set low, toggle or disable.
4. Set up HW channel, including clock source and prescaler.
   - Select a master bus working in MCB mode.
   - If not, OcuClockSource won't take effect.
5. Enable OCU ISR and channel used.
   - If not, GPT channel cannot work.
   - This enablement doesn't set FEN bit.

Legend

Tab n No. Note

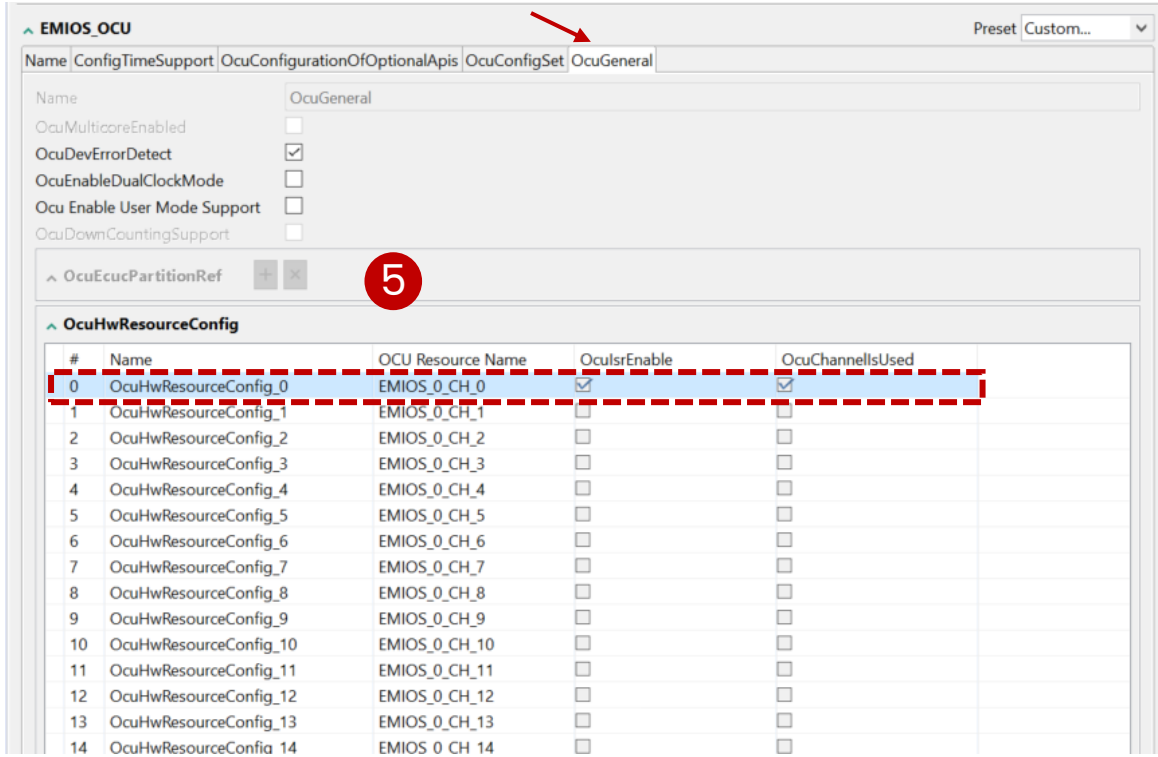


Emios_Ocu (Continued)

eMIOS Configuration Using RTD 3.0.0 (LLD)

- **OCU – General process in ConfigTools**

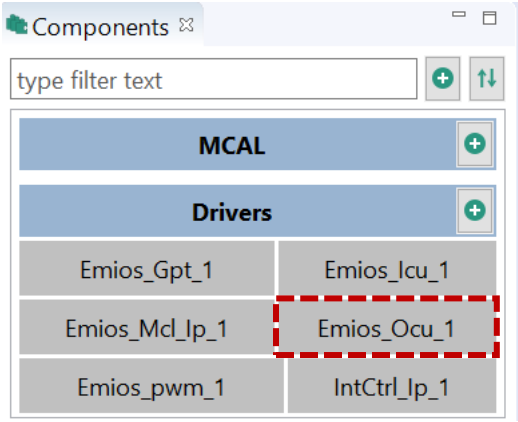
1. Enable optional APIs if needed.
2. Configure OCU channel and default threshold and period.
   - The selected channel must be added in OcuHWSpecificSettings.
   - The default threshold should smaller than max counter value (period).
3. Configure pin default state and pin action.
   - Set high, set low, toggle or disable.
4. Set up HW channel, including clock source and prescaler.
   - Select a master bus working in MCB mode.
   - If not, OcuClockSource won't take effect.
5. Enable OCU ISR and channel used.
   - If not, GPT channel cannot work.
   - This enablement doesn't set FEN bit.





Emios_Ocu (Continued)

eMIOS Configuration Using RTD 3.0.0 (LLD)



- **Use tips:**
  - Function: configure Output Compare Unit (OCU) using eMIOS channel.
  - SAOC and GPI modes' support.
  - Call **Emios_Ocu_Ip_EnableNotification**() after OCU channel initialization, if needed.
  - The OCU starts after calling **Emios_Ocu_Ip_StartChannel**().
  - **Emios_Ocu_Ip_IrqHandler**() is in Emios_Ocu_Ip_Irq.c.
  - The OCU channel must select a master bus as time base and local counter cannot be selected.
- **Key APIs[1]:**
  - void **Emios_Ocu_Ip_Init**(const Emios_Ocu_Ip_ModuleConfigType * const pEmiosIpConfig)
  - void **Emios_Ocu_Ip_EnableNotification**(uint8 InstNum, uint8 ChNum)
  - void **Emios_Ocu_Ip_StartChannel**(uint8 InstNum, uint8 ChNum)
    - Start OCU by entering SAOC mode.
  - void **Emios_Ocu_Ip_StopChannel**(uint8 InstNum, uint8 ChNum)
    - Stop OCU by going back to GPI mode.
  - void **Emios_Ocu_Ip_SetPinState**(uint8 InstNum, uint8 ChNum, Emios_Ocu_Ip_PinStateType PinState)
  - void **Emios_Ocu_Ip_SetPinAction**(uint8 InstNum, uint8 ChNum, Emios_Ocu_Ip_PinActionType PinAction)
    - Set high, set low, toggle or disable.

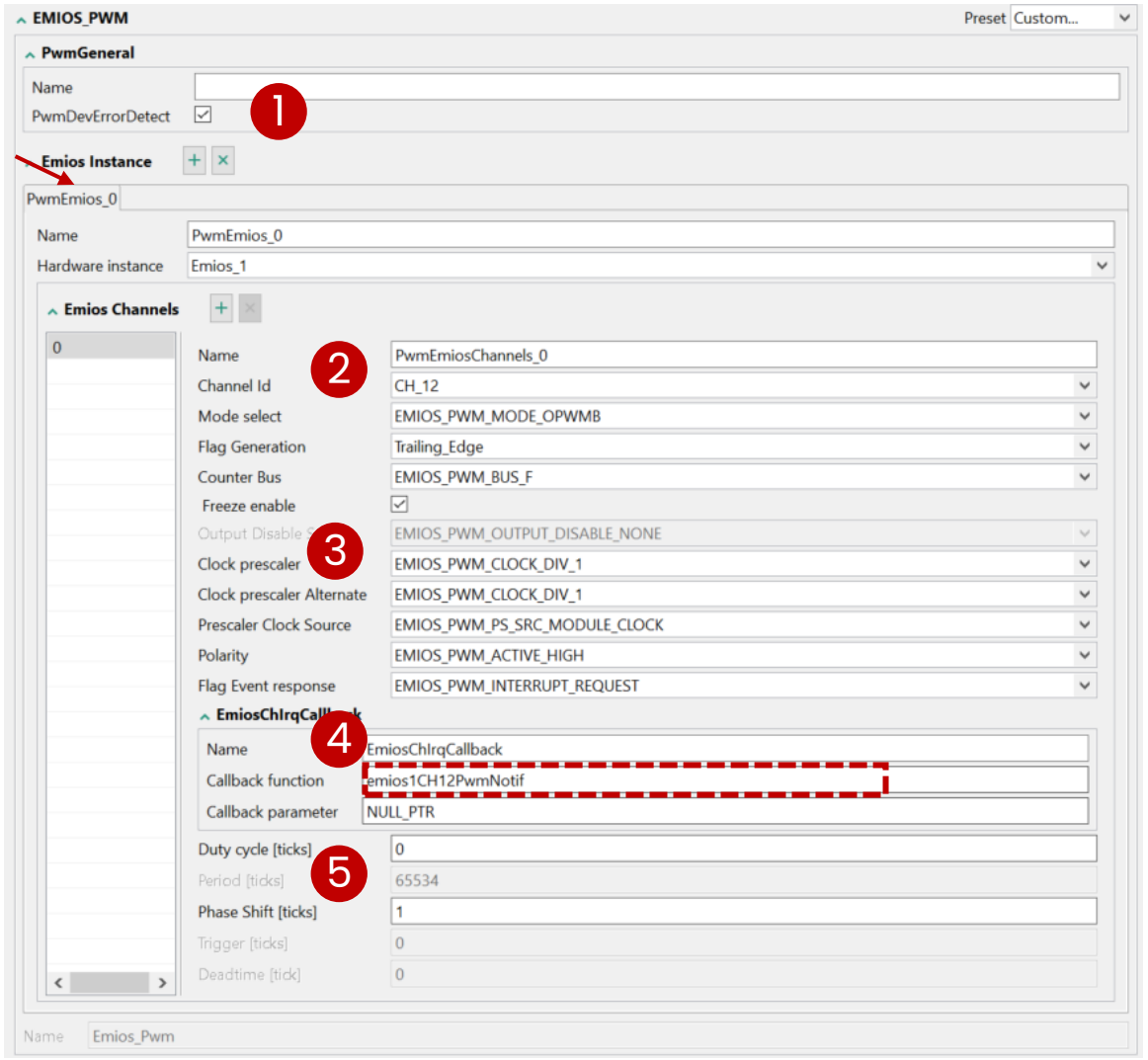
[1] Not all the API is listed here.

Emios_Pwm

eMIOS Configuration Using RTD 3.0.0 (LLD)

- **PWM – General process in ConfigTools**

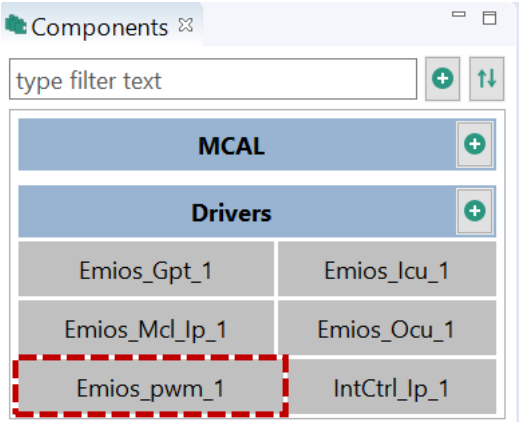
1. Enable error detect.
   - If not, channel cannot entry freeze mode (debug mode).
2. Configure HW channel and PWM mode.
   - DAOC, OPWFMB, OPWMCB, OPWMB and OPWMT modes support.
   - Select flag generation position: trailing edge, or both leading and trailing edges.
   - Master bus should be configured in MCL.
3. Set channel prescaler and polarity.
   - If you're using master bus as time base, local prescaler won't take effect.
4. Set callback function.
5. Set initial duty and shift.
   - For OPWMT mode, shift cannot set to 0.





Emios_Pwm (Continued)

eMIOS Configuration Using RTD 3.0.0 (LLD)



- **Use tips:**
  - Function: configure Pulse Width Modulation (PWM) using eMIOS channel.
  - Limitation: PWM channels cannot start simultaneously if it's not in common time base.
  - DAOC, OPWFMB, OPWMCB, OPWMB, OPWMT and GPO modes' support.
    - DAOC is a one-pulse PWM; update A/B registers every period for continuous pulses.
  - If 0% or 100% duty cycle is required:
    - Interrupt will be disabled. Global variable *Emios_Pwm_Ip_aNotif* of corresponding channel will be set to 1.
    - Restoration operation of Interrupt and *Emios_Pwm_Ip_aNotif* will be done if set to other duty.
  - Call **Emios_Pwm_Ip_SetFlagRequest**() after PWM channel initialization if notification needed.
    - You may need to check *Emios_Pwm_Ip_aNotif* if initial duty is 0%.
  - Call **Emios_Pwm_Ip_SetOutputState**() to stop PWM and call **Emios_Pwm_Ip_SetOutputToNormal**() to resume PWM.
  - For PWM in OPWMT mode, set duty cycle to (period - phase shift) to reach 100% duty.
    - Setting duty cycle equal to period or 0 results in 0% duty cycle.
  - For other PWM modes, set duty cycle to 0 to get 0% duty and set duty cycle to period to get 100% duty.
  - Because DAOC can only generate a single pulse, to use DAOC to generate a continuous PWM waveform you need to call **Emios_Pwm_Ip_IrqDaocHandler**() periodically, so the DAOC interrupt must be turned on.
  - If a PWM using master timer bus needs to be changed with another period, call **Emios_Mcl_Ip_SetCounterBusPeriod**()

Emios_Pwm (Continued)

eMIOS Configuration Using RTD 3.0.0 (LLD)

- **Key APIs[1]:**
  - void **Emios_Pwm_Ip_InitChannel**(uint8 instance, Emios_Pwm_Ip_ChannelConfigType const *userChCfg)
  - void **Emios_Pwm_Ip_SetFlagRequest**(uint8 instance, uint8 channel, Emios_Pwm_Ip_InterruptType event)
  - void **Emios_Pwm_Ip_SetOutputState**(uint8 instance, uint8 channel, Emios_Pwm_Ip_OutputStateType outputState)
    - Enter GPO mode and disable channel interrupts.
  - void **Emios_Pwm_Ip_SetOutputToNormal**(uint8 instance, uint8 channel, uint16 dutyPercent, Emios_Pwm_Ip_PolarityType polarity, Emios_Pwm_Ip_PwmModeType mode)
    - Back to specific mode, not previous mode.
  - void **Emios_Pwm_Ip_SetPeriod**(uint8 instance, uint8 channel, uint16 newPeriod)
    - Set PWM using internal counter to a new period. Support for DAOC and OPWFMB only. Call Emios_Mcl_Ip_SetCounterBusPeriod() for PWM modes using the master timer bus.
  - Emios_Pwm_Ip_StatusType **Emios_Pwm_Ip_SetDutyCycle**(uint8 instance, uint8 channel, uint16 newDutyCycle)
    - If 0% or 100% duty cycle is desired, the interrupt will be disabled.
  - Emios_Pwm_Ip_StatusType **Emios_Pwm_Ip_SetPhaseShift**(uint8 instance, uint8 channel, uint16 phaseShift)
  - void **Emios_Pwm_Ip_SetDeadTime**(uint8 instance, uint8 channel, uint16 newDeadTime)
  - void **Emios_Pwm_Ip_SetTriggerPlacement**(uint8 instance, uint8 channel, uint32 newTriggerPlacement)
    - Set trigger placement for OPWMT mode.

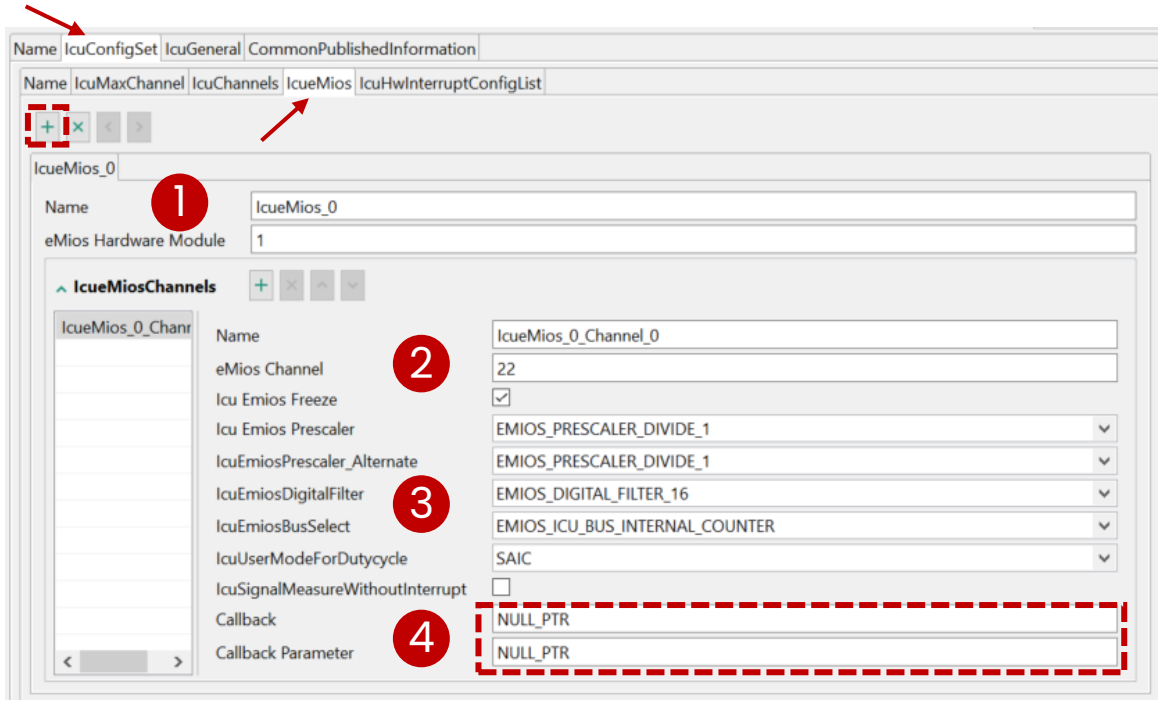
[1] Not all the API is listed here.

Emios_Icu

eMIOS Configuration Using RTD 3.0.0 (LLD)

- **ICU – General process in ConfigTools**

1. Configure eMIOS instance to be used.
2. Configure HW channel.
   - The channel must be enabled in *IcuHwInterruptConfigList*.
3. Config channel prescaler and filter and select user mode for duty cycle.
   - Overflow notification not support internal counter.
   - Prescaler is up to 4.
   - Digital filter is up to 16.
   - User mode for duty cycle support SAIC and IPWM.
4. Enable measure without interrupt if needed.





Emios_Icu (Continued)

eMIOS Configuration Using RTD 3.0.0 (LLD)

- **ICU – General process in ConfigTools**

5 Select a HW channel reference for an ICU channel configured in *IcueMios*.

- Set the measurement mode:
  ICU_MODE_SIGNAL_EDGE_DETECT
  ICU_MODE_EDGE_COUNTER
  ICU_MODE_SIGNAL_MEASUREMENT ICU_MODE_TIMESTAMP

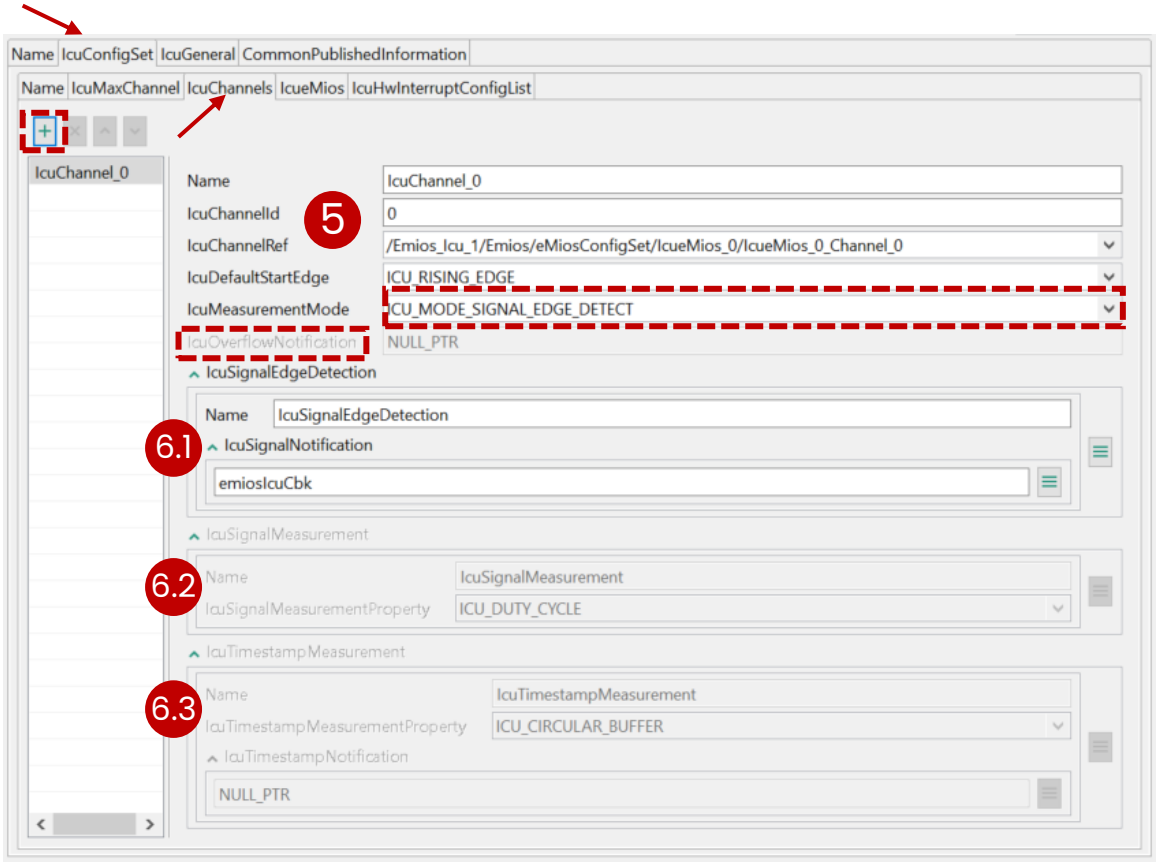
6 Set the mode configuration according to selected mode:

6.1 Set ICU signal notification if edge detect is selected.

6.2 Set measurement property, ICU_DUTY_CYCLE, ICU_HIGH_TIME, ICU_LOW_TIME, ICU_PERIOD_TIME if signal measurement is selected.

6.3 Set timestamp property, ICU_CIRCULAR_BUFFER or ICU_LINEAR_BUFFER, and notification, if timestamp is selected.

- **Note**: The overflow notification is applied for signal measurement and edge counter.



Legend

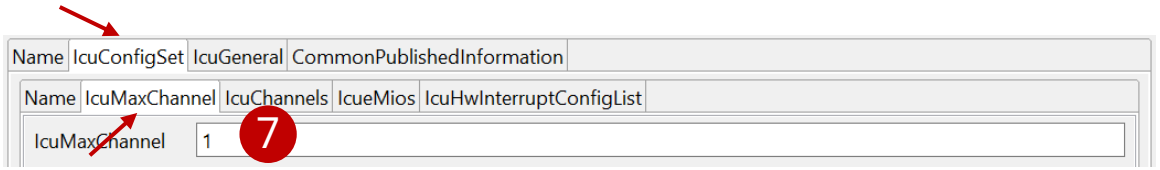
Tab No. Note

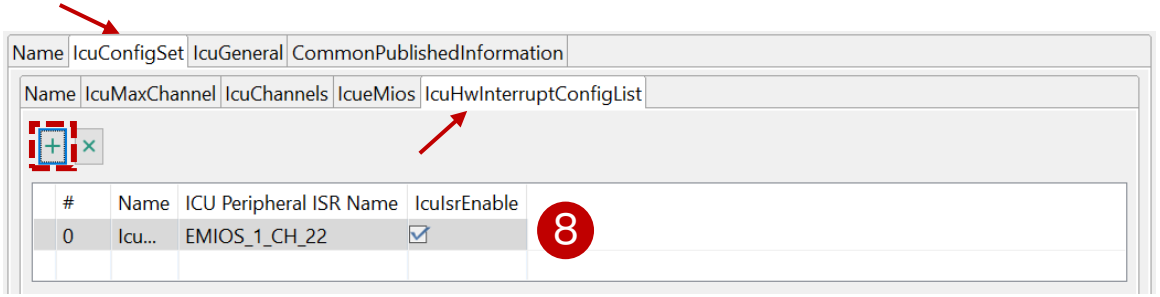
Emios_Icu (Continued)

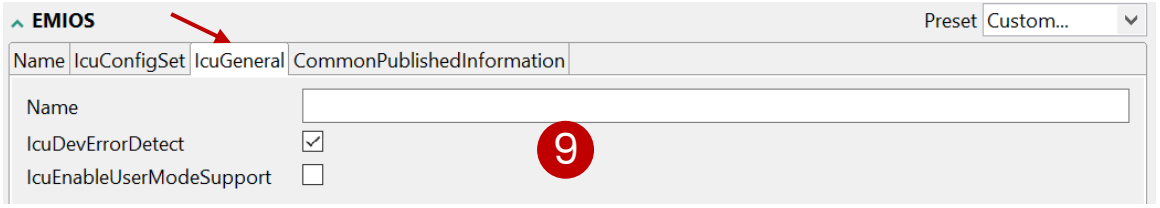
eMIOS Configuration Using RTD 3.0.0 (LLD)

- **ICU – General process in ConfigTools**

7. *IcuMaxChannel* must equal the number of configured ICU channels.
8. Enable the channel to be used.
   - There will be a warning otherwise.
9. Enable error detect if needed.



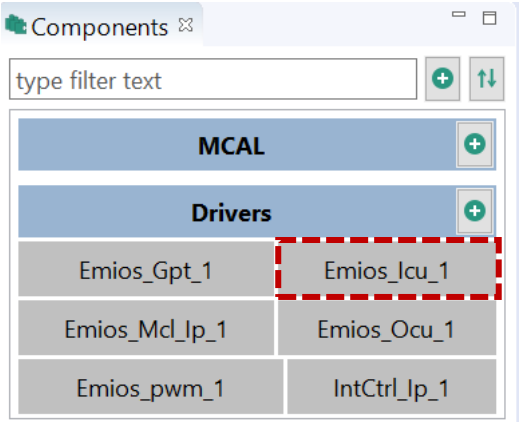






Emios_Icu (Continued)

eMIOS Configuration Using RTD 3.0.0 (LLD)



- **Use tips:**
  - Function: configure Input Capture Unit (ICU) using eMIOS channel.
  - IPWM, IPM, SAIC, PEC and GPI modes' support.
  - There are four ICU modes:
    - ICU_MODE_SIGNAL_EDGE_DETECT
    - ICU_MODE_EDGE_COUNTER
    - ICU_MODE_SIGNAL_MEASUREMENT
    - ICU_MODE_TIMESTAMP
  - Call following APIs to start ICU after initialization:
    - **Emios_Icu_Ip_EnableEdgeDetection**(), **Emios_Icu_Ip_StartTimestamp**(), **Emios_Icu_Ip_EnableEdgeCount**(), **Emios_Icu_Ip_StartSignalMeasurement**().
- **Key APIs[1]:**
  - eMios_Icu_Ip_StatusType **Emios_Icu_Ip_InitChannel**(uint8 instance, const eMios_Icu_Ip_ConfigType *pIcParam)
  - void **Emios_Icu_Ip_EnableEdgeDetection**(const uint8 u8ModuleIdx, const uint8 u8ChannelIdx)
  - void **Emios_Icu_Ip_StartTimestamp**(const uint8 u8ModuleIdx, const uint8 u8ChannelIdx, uint16 * pBufferPtr, uint16 u16BufferSize, uint16 u16NotifyInterval)
  - void **Emios_Icu_Ip_EnableEdgeCount**(const uint8 u8ModuleIdx, const uint8 u8ChannelIdx)
  - void **Emios_Icu_Ip_StartSignalMeasurement**(const uint8 u8ModuleIdx, const uint8 u8ChannelIdx)

[1] Not all the API is listed here.

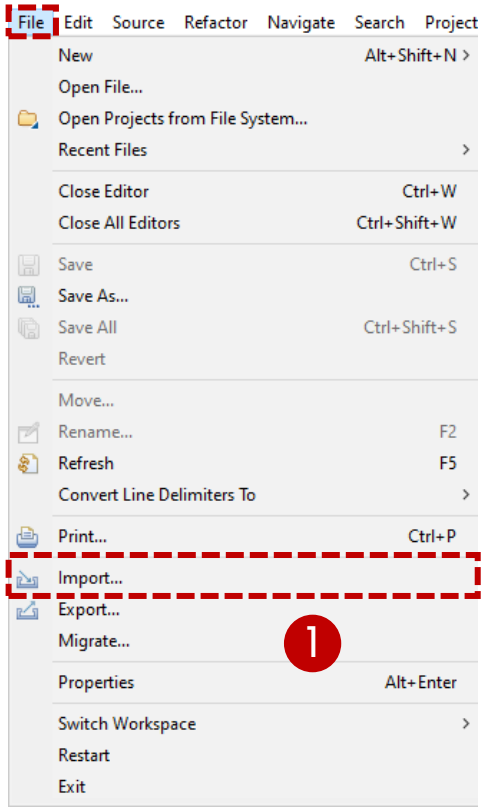
04

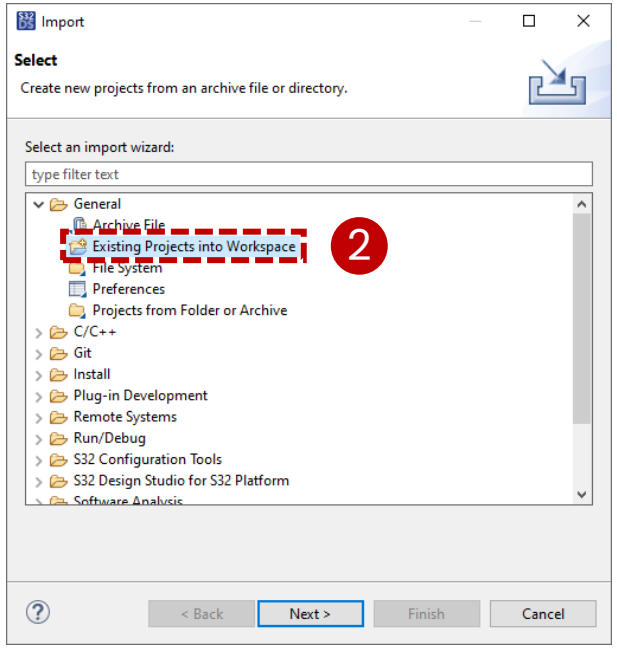
Import and Run S32DS RTD 3.0.0 Examples



Import and Run S32DS RTD 3.0.0 eMIOS Example Projects

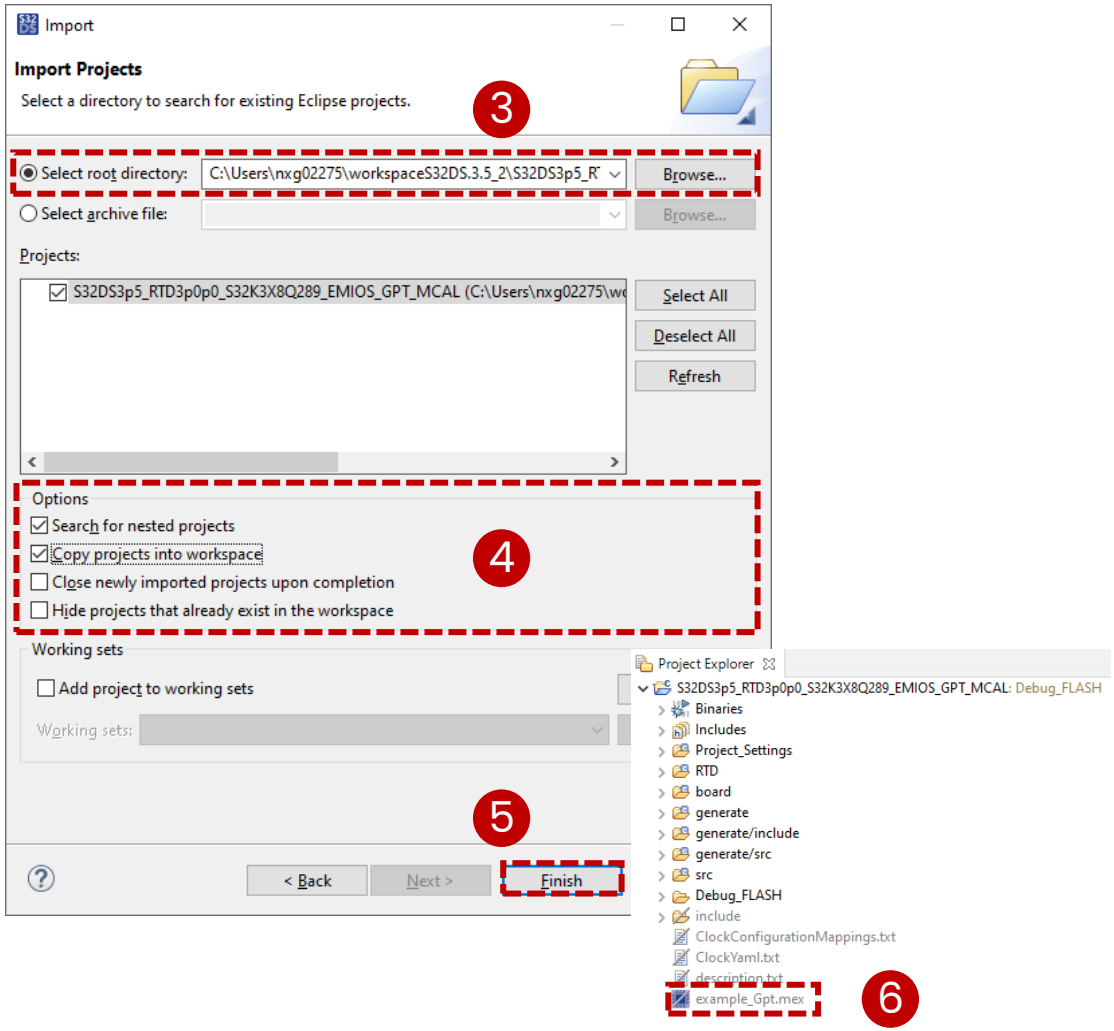
1. Click on File → Import….
2. Select General → Existing Projects into Workspace.
3. Select root directory and choose the project according to the device and eMIOS mode you need (GPT, OCU, PWM, ICU).
4. In Options panel, tick checkbox Search for nested projects, and optionally you can tick Copy projects into workspace.
5. Click Finish.
6. Double click on .mex file.
   - When you double click on the .mex file there is a possibility that you will get the following warning: “Loaded configuration has been created by older version of the tool”. If so, click OK.





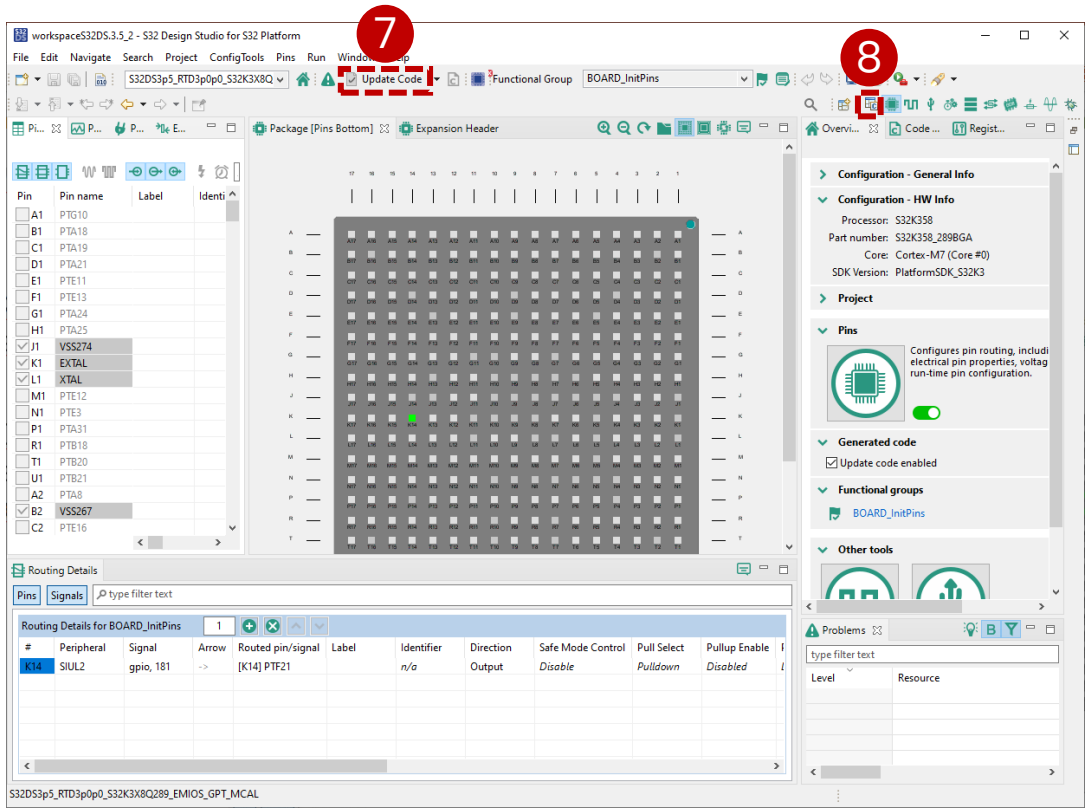
Import and Run S32DS RTD 3.0.0 eMIOS Example Projects (Continued)

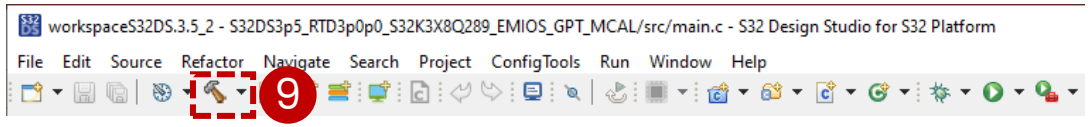
1. Click on File → Import….
2. Select General → Existing Projects into Workspace.
3. Select root directory and choose the project according to the device and eMIOS mode you need (GPT, OCU, PWM, ICU).
4. In Options panel, tick checkbox Search for nested projects, and optionally you can tick Copy projects into workspace.
5. Click Finish.
6. Double click on .mex file.
   - When you double click on the .mex file there is a possibility that you will get the following warning: “Loaded configuration has been created by older version of the tool”. If so, click OK.



Import and Run S32DS RTD 3.0.0 eMIOS Example Projects (Continued)

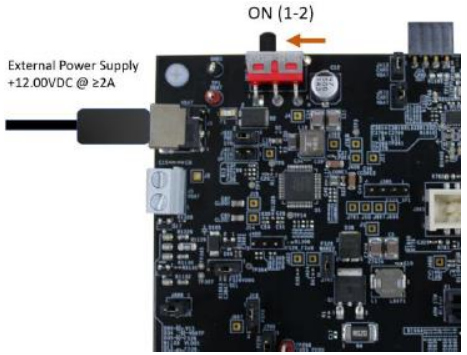
7. Click on Update Code.
8. Once the project has been updated, switch to C/C++ perspective in the IDE.
9. Build the project.
10. Connect power supply and micro-USB cable to the Open SDA connector (e.g., J55 on S32K358) to the EVB.
11. Click on the Debug icon.
12. Select the *_FLASH_PNE option in configurator launcher.
13. Run the example.



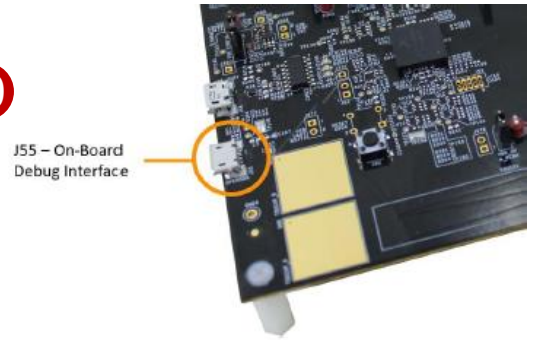


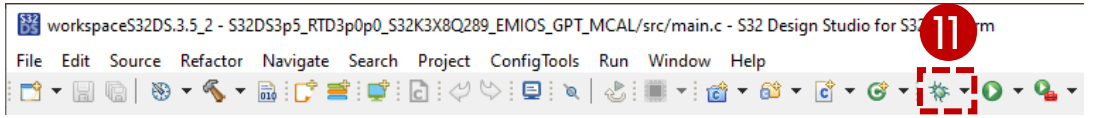
Import and Run S32DS RTD 3.0.0 eMIOS Example Projects (Continued)

7. Click on Update Code.
8. Once the project has been updated, switch to C/C++ perspective in the IDE.
9. Build the project.
10. Connect power supply and micro-USB cable to the Open SDA connector (e.g., J55 on S32K358) to the EVB.
11. Click on the Debug icon.
12. Select the *_FLASH_PNE option in configurator launcher.
13. Run the example.

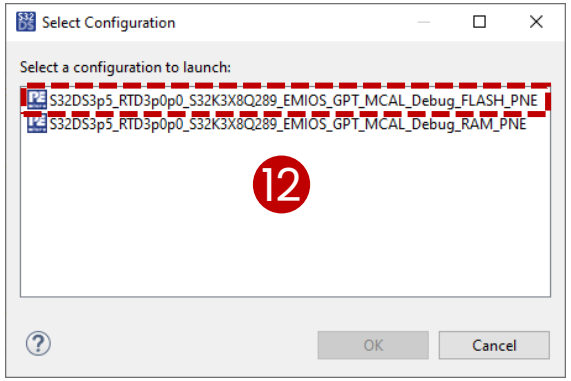


10





11



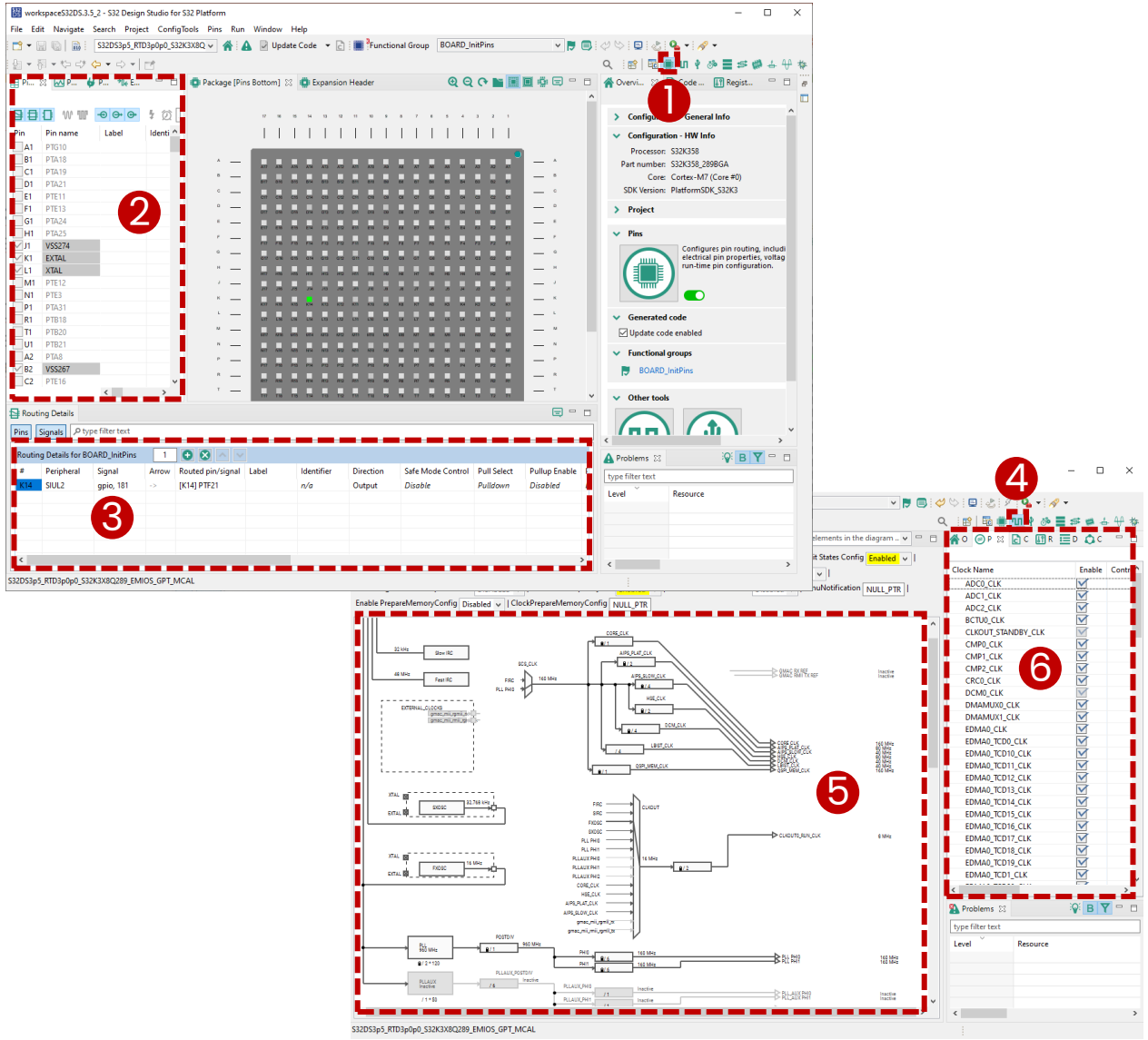
12



13

S32DS RTD 3.0.0 eMIOS Example Projects – Pins and Clocks Views

1. Switch to the Pins perspective.
2. You can check which pins are in use in the example in the left panel, as well as all their alternate functions.
3. In the Routing Details panel, you can select different options for each pin in use, such as Direction, Slew Rate, give it a Label, etc.
4. Switch to the Clocks perspective.
5. The Clocks Diagram will display on the main panel.
6. In the Peripheral Clock View panel, you can select which clocks to enable, their source and divider, and see at which frequency said clock will run.



05

S32DS RTD 3.0.0 eMIOS Examples (LLD)



eMIOS Example Project: GPT (LLD)

S32DS RTD 3.0.0 eMIOS Examples (LLD)

- **Description:**
  - Use eMIOS GPT to generate periodic timer interrupt and Siul2_Dio driver to toggle LED on/off periodically.
- **Dependent RTD functions:**
  - **Clock_Ip_Init**:
    - Initialize the clock sources, the clock tree and to configure the clock gating of the peripherals.
  - **Siul2_Port_Ip_Init**:
    - Initializes the pins used by the application using the Siul2_Port API.
  - **IntCtrl_Ip_EnableIrq, IntCtrl_Ip_InstallHandler**:
    - Enable the interrupt used by the application and set the correct handler for it.
  - **Emios_Mcl_Ip_Init**:
    - Initialize eMIOS counter bus used by GPT.
  - **Emios_Gpt_Ip_InitChannel, Emios_Gpt_Ip_EnableChannelInterrupt, Emios_Gpt_Ip_StartTimer**:
    - Initialize the GPT driver, start the selected channel and enable its notification.
- **HW requirements:**
  - S32K312EVB-Q172 (schematic: SPF-50892 Rev.B)
  - S32K3X4EVB-S172 (schematic: SPF-51972 Rev.A)
  - S32K3X8EVB-Q289 (schematic: SPF-54870 Rev.C)

eMIOS Example Project: GPT (LLD) (Continued)

S32DS RTD 3.0.0 eMIOS Examples (LLD)

- **eMIOS GPT example:**
  - main.c:

Component	Description	Channel (EVB)
Emios_Mcl_Ip	Initialize instance and master bus	eMIOS_0_CH23
Emios_Gpt	Generate periodic interrupt	eMIOS_0_CH0

```c
/* Includes */
#include "Emios_Gpt_Ip.h"
#include "Emios_Mcl_Ip.h"
#include "Emios_Mcl_Ip_Irq.h"
#include "Clock_Ip.h"
#include "IntCtrl_Ip.h"
#include "Siul2_Port_Ip.h"
#include "Siul2_Dio_Ip.h"

/* Local macros */
#define clockConfig &Clock_Ip_aClockConfig[0]
#define EMIOS_0       (0U)
#define CHANNEL_0     (0U)

/* Global flag updated in IRQ */
static volatile uint8 toggleLed = 0U;

/* EMIOS notification periodically called by
configured channel used to blink LED */
void GptChannel_0_notification()
{
    toggleLed = 1U;
}
```

```c
/* Init clock */
Clock_Ip_Init(clockConfig);
/* Initialize all pins using the Siul2_Port driver */
Siul2_Port_Ip_Init(NUM_OF_CONFIGURED_PINS0,g_pin_mux_InitConfigArr0);
/* Install EMIOS ISR */
IntCtrl_Ip_InstallHandler(EMIOS0_5_IRQn, &EMIOS0_5_IRQ, NULL_PTR);
IntCtrl_Ip_EnableIrq(EMIOS0_5_IRQn);
/* Init eMIOS MCL for enable global counter bus */
Emios_Mcl_Ip_Init(EMIOS_0, &Emios_Mcl_Ip_0_Config_VS_0);
/* Init eMIOS GPT */
Emios_Gpt_Ip_InitChannel(EMIOS_0, &EMIOS_0_ChannelConfig_PB_VS_0[0]);
/* Enable eMIOS GPT channel notification */
Emios_Gpt_Ip_EnableChannelInterrupt(EMIOS_0, CHANNEL_0);
/* Start eMIOS GPT Channel */
Emios_Gpt_Ip_StartTimer(EMIOS_0, CHANNEL_0, 65533);
while (1)
{
    /* Toggle the LED when the GPT notification is called */
    if (1U == toggleLed)
    {
        toggleLed = 0;
        Siul2_Dio_Ip_WritePin(LED_PORT, LED_PIN, stateLed);
        stateLed = stateLed ^ 1U;
    }
}
```

eMIOS Example Project: OCU (LLD)

S32DS RTD 3.0.0 eMIOS Examples (LLD)

- **Description:**
  - Initialize eMIOS channel. Through OCU a notification will be generated upon each compare match and a loop will run to blink an LED.
    - Notification will toggle a pin and increase 'count' variable. The loop will toggle LED every 10 increments of the counter using Siul2_Dio driver.
- **Dependent RTD functions:**
  - **Clock_Ip_Init**:
    - Initialize the clock sources, the clock tree and to configure the clock gating of the peripherals.
  - **Siul2_Port_Ip_Init**:
    - Initializes the pins used by the application using the Siul2_Port API.
  - **IntCtrl_Ip_EnableIrq, IntCtrl_Ip_InstallHandler**:
    - Enable the interrupt used by the application and set the correct handler for it.
  - **Emios_Mcl_Ip_Init**:
    - Initialize eMIOS counter bus used by OCU.
  - **Emios_Ocu_Ip_Init, Emios_Ocu_Ip_StartChannel, Emios_Ocu_Ip_EnableNotification**:
    - Initialize the OCU driver, start the selected channel and enable its notification.
- **HW requirements:**
  - S32K312EVB-Q172 (schematic: SPF-50892 Rev.B)
  - S32K3X4EVB-S172 (schematic: SPF-51972 Rev.A)
  - S32K3X8EVB-Q289 (schematic: SPF-54870 Rev.C)

eMIOS Example Project: OCU (LLD) (Continued)

S32DS RTD 3.0.0 eMIOS Examples (LLD)

- **eMIOS OCU example:**
  - main.c:

Component	Description	Channel (EVB)
Emios_Mcl_IP	Initialize instance and master bus	eMIOS_0_CH0
Emios_Ocu	Toggle pin state	eMIOS_0_CH1

```
/* Includes */
#include "Emios_Ocu_Ip.h"
#include "Emios_Mcl_Ip.h"
#include "Emios_Mcl_Ip_Irq.h"
#include "Clock_Ip.h"
#include "IntCtrl_Ip.h"
#include "Siul2_Port_Ip.h"
#include "Siul2_Dio_Ip.h"
/* Local macros */
#define clockConfig &Clock_Ip_aClockConfig[0]
#define EMIOS_0       (0U)
#define CHANNEL_1     (1U)
/* Global flag updated in IRQ */
static volatile uint8 toggleLed = 0U;
/* EMIOS notification periodically called.
Used to blink LED */
void OcuChannel_0_notification()
{
    static uint16 count = 0U;
    if(++count >= 10U)
    {
        toggleLed = 1U;
        count = 0;
    }
}
```

```
/* Init clock */
Clock_Ip_Init(clockConfig);
/* Initialize all pins using the Siul2_Port driver */
Siul2_Port_Ip_Init(NUM_OF_CONFIGURED_PINS0,g_pin_mux_InitConfigArr0);
/* Set EMIOS interrupt */
IntCtrl_Ip_Init(&IntCtrlConfig_0);
IntCtrl_Ip_EnableIrq(0U);
/* Init EMIOS MCL for enable global counter bus */
Emios_Mcl_Ip_Init(EMIOS_0, &Emios_Mcl_Ip_0_Config_VS_0);
/* Init EMIOS OCU */
Emios_Ocu_Ip_Init(&Emios_Ocu_Ip_0_ModuleCfgPB_VS_0);
/* Enable EMIOS OCU Channel notification */
Emios_Ocu_Ip_EnableNotification(EMIOS_0,CHANNEL_1);
/* Start EMIOS OCU Channel */
Emios_Ocu_Ip_StartChannel(EMIOS_0,CHANNEL_1);
while (1)
{
    /* Toggle the LED when the OCU notification is called */
    if (1U == toggleLed)
    {
        toggleLed = 0;
        Siul2_Dio_Ip_WritePin(LED_PORT, LED_PIN, stateLed);
        stateLed = stateLed ^ 1U;
    }
}
```

eMIOS Example Project: PWM (LLD)

S32DS RTD 3.0.0 eMIOS Examples (LLD)

- **Description:**
  - PWM example using eMIOS to turn on LED and regulate its brightness.
- **Dependent RTD functions:**
  - **Clock_Ip_Init**:
    - Initialize the clock sources, the clock tree and to configure the clock gating of the peripherals.
  - **Siul2_Port_Ip_Init**:
    - Initializes the pins used by the application using the Siul2_Port API.
  - **Emios_Mcl_Ip_Init**:
    - Initialize eMIOS counter bus used by PWM.
  - **Emios_Pwm_Ip_InitChannel, Emios_Mcl_Ip_SetCounterBusPeriod, Emios_Pwm_Ip_SetDutyCycle**:
    - Initialize the PWM channel, change the period and duty cycle to control the led brightness and blink LED.
- **HW requirements:**
  - S32K312EVB-Q172 (schematic: SPF-50892 Rev.B)
  - S32K3X4EVB-S172 (schematic: SPF-51972 Rev.A)
  - S32K3X8EVB-Q289 (schematic: SPF-54870 Rev.C)

eMIOS Example Project: PWM (LLD) (Continued)

S32DS RTD 3.0.0 eMIOS Examples (LLD)

- **eMIOS PWM example:**
  - main.c:

Component	Description	Channel (EVB)
Emios_Mcl_Ip	Initialize instance and master bus	eMIOS_1_CH23
Emios_Pwm	Generate OPWMB PWM signal	eMIOS_1_CH12

```c
/* Includes */
#include "Clock_Ip.h"
#include "Emios_Pwm_Ip.h"
#include "Emios_Mcl_Ip.h"
#include "Siul2_Port_Ip.h"

/* Local macros */
#define NUM_BLINK_LED      (uint32)10U
#define DELAY_TIMER        (uint32)5000000U
#define INSTANCE_1         (uint8)1U
#define CHANNEL_12         (uint8)12U
#define CHANNEL_23         (uint8)23U

/* Local functions */
void TestDelay(uint32 delay)
{
    static volatile uint32 DelayTimer = 0;
    while(DelayTimer<delay)
    {
        DelayTimer++;
    }
    DelayTimer=0;
}
```

```c
/* Initialize clock */
Clock_Ip_Init(&Clock_Ip_aClockConfig[0]);
/* Initialize all pins using the Port driver */
Siul2_Port_Ip_Init(NUM_OF_CONFIGURED_PINS0, g_pin_mux_InitConfigArr0);
/* Initialize EMIOS MCL driver */
Emios_Mcl_Ip_Init(INSTANCE_1, &Emios_Mcl_Ip_1_Config_VS_0);
/* Initialize EMIOS PWM driver */
Emios_Pwm_Ip_InitChannel(EMIOS_PWM_IP_VS_0_I1_CH12_CFG, &Emios_Pwm_Ip_VS_0_I1_Ch12);
/* Set new period for the channels that used external counter bus */
Emios_Mcl_Ip_SetCounterBusPeriod(INSTANCE_1, CHANNEL_23, 0x5000);
/* Using duty 0% and 100% to blink LED */
while(num_blink < NUM_BLINK_LED)
{
    /* LED on when duty cycle is 100% */
    Emios_Pwm_Ip_SetDutyCycle(INSTANCE_1, CHANNEL_12, 0x5000);
    TestDelay(DELAY_TIMER);
    /* LED off when duty cycle is 0% */
    Emios_Pwm_Ip_SetDutyCycle(INSTANCE_1, CHANNEL_12, 0U);
    TestDelay(DELAY_TIMER);
    num_blink++;
}
/* De-Initialize EMIOS PWM driver */
Emios_Pwm_Ip_DeInitChannel(INSTANCE_1, CHANNEL_12);
```

eMIOS Example Project: ICU (LLD)

S32DS RTD 3.0.0 eMIOS Examples (LLD)

- **Description:**
  - GPIO trigger will generate edges to ICU input pins. Interrupts will be generated for the ICU channels and the user function will be called to blink the LED.
- **Dependent RTD functions:**
  - **Clock_Ip_Init, Power_Ip_SetMode**:
    - Initialize the clock sources, the clock tree and to configure the clock gating of the peripherals.
  - **IntCtrl_Ip_EnableIrq, IntCtrl_Ip_InstallHandler**:
    - Install eMIOS ISR.
  - **Siul2_Port_Ip_Init, Siul2_Dio_Ip_WritePin**:
    - Initialize all pins using the Siul2_Port driver, generate edges to ICU input pins, blink LED.
  - **Emios_Icu_Ip_Init, Emios_Icu_Ip_EnableEdgeDetection, Emios_Icu_Ip_EnableNotification**:
    - Initialize, enable edge detection and notification of ICU driver.
- **HW requirements:**
  - S32K312EVB-Q172 (schematic: SPF-50892 Rev.B)
  - S32K3X4EVB-S172 (schematic: SPF-51972 Rev.A)
  - S32K3X8EVB-Q289 (schematic: SPF-54870 Rev.C)

eMIOS Example Project: ICU (LLD)

S32DS RTD 3.0.0 eMIOS Examples (LLD)

- **eMIOS ICU example:**
  - main.c:

Component	Description	Channel (EVB)
Emios_Mcl_Ip	Initialize instance and master bus	eMIOS_1_CH23
Emios_Icu	Rising edge detection	eMIOS_1_CH1

```
/* Global functions */
extern ISR(EMIOS0_5_IRQ);
/* Blink LED using user function */
void User_EdgeDetect0(void) {
    /* Increment IRQ counter */
    UserCountIrqCH0++;
    if(UserCountIrqCH0 % 2 == 0) {
Siul2_Dio_Ip_WritePin(Digital_Output_LED_RED_P
ORT, Digital_Output_LED_RED_PIN, 1);
    }
    else {
Siul2_Dio_Ip_WritePin(Digital_Output_LED_RED_P
ORT, Digital_Output_LED_RED_PIN, 0);
    }
}
/* Initialize clock */
Clock_Ip_InitClock(&Clock_Ip_aClockConfig[0]);
Power_Ip_SetMode(&Power_Ip_aModeConfigPB[0]);
Siul2_Port_Ip_Init(NUM_OF_CONFIGURED_PINS0,
g_pin_mux_InitConfigArr0);
```

```
/* Install EMIOS ISR */
IntCtrl_Ip_InstallHandler(EMIOS1_5_IRQn, &EMIOS1_5_IRQ, NULL_PTR);
IntCtrl_Ip_EnableIrq(EMIOS1_5_IRQn);
/* Initialize the ICU driver */
Emios_Icu_Ip_Init(1,&eMios_Icu_Ip_1_Config_PB_VS_0);
/* Enable ICU edge detection and notification */
Emios_Icu_Ip_EnableEdgeDetection(1,1);
Emios_Icu_Ip_EnableNotification(1,1);
/* Generate input for ICU channel */
while (TRUE) {
  if(BlinkCount % 2 == 0) {
      Siul2_Dio_Ip_WritePin(Port_pin_InputForIcuChannel0_PORT,Port_pin_InputForIcuChannel0_PIN,1);
      delay();
  }
  else {
      Siul2_Dio_Ip_WritePin(Port_pin_InputForIcuChannel0_PORT,Port_pin_InputForIcuChannel0_PIN,0);
      delay();
  }
  BlinkCount++;
}
```

06

S32DS RTD 3.0.0 eMIOS Examples (MCAL)



eMIOS Example Project: GPT (MCAL)

S32DS RTD 3.0.0 eMIOS Examples (MCAL)

- **Description:**
  - Use eMIOS GPT to generate periodic timer interrupt and DIO driver to toggle LED on/off periodically.
- **Dependent RTD functions:**
  - **Mcu_Init, Mcu_InitClock, Mcu_SetMode**:
    - Initialize the clock sources, the clock tree and to configure the clock gating of the peripherals.
  - **Port_Init**:
    - Initialize all pins.
  - **Platform_Init, Mcl_Init**:
    - Initialize eMIOS interrupt and eMIOS common.
  - **Gpt_Init, Gpt_EnableNotification, Gpt_StartTimer, Dio_WriteChannel**:
    - Initialize the GPT driver, enable its notification and start timer. Use DIO driver to toggle LED on/off.
- **HW requirements:**
  - S32K3X8EVB-Q289 (schematic: SPF-54870 Rev.C)
- **For further information regarding the EMIOS GPT RTD driver, review the documentation in the following path (for default RTD installation):**
  - C:\NXP\S32DS.3.5\S32DS\software\PlatformSDK_S32K3\RTD\Gpt_TS_T40D34M30I0R0\doc

eMIOS Example Project: GPT (MCAL) (Continued)

S32DS RTD 3.0.0 eMIOS Examples (MCAL)

- **eMIOS GPT example:**
  - main.c:

Component	Description	Channel (EVB)
MCL	Initialize instance and master bus	eMIOS_0_CH23
GPT	Generate periodic interrupt	eMIOS_0_CH0

```c
/* Includes */
#include "Gpt.h"
#include "Port.h"
#include "Dio.h"
#include "Mcu.h"
#include "Platform.h"
#include "Mcl.h"

/* Global flag updated in IRQ */
static volatile uint8 toggleLed = 0U;

/* EMIOS notification periodically called
by configured channel used to blink LED */
void GptChannel_0_notification()
{
    toggleLed = 1U;
}
```

```c
/* Initialize the MCU driver */
Mcu_Init(&Mcu_Config_VS_0);
/* Initialize the clock tree */
Mcu_InitClock(McuClockSettingConfig_0);
/* Apply a mode configuration */
Mcu_SetMode(McuModeSettingConf_0);
/* Initialize all pins using the Port driver */
Port_Init(&Port_Config_VS_0);
/* Set eMIOS interrupt */
Platform_Init(NULL_PTR);
/* Initialize MCL driver */
Mcl_Init(&Mcl_Config_VS_0);
/* Initialize the high-level configuration structure of GPT driver */
Gpt_Init(&Gpt_Config_VS_0);
/* Enable eMIOS GPT channel notification */
Gpt_EnableNotification(GptConf_GptChannelConfiguration_GptChannelConfiguration_0);
/* Start eMIOS GPT Channel */
Gpt_StartTimer(GptConf_GptChannelConfiguration_GptChannelConfiguration_0, 60000U);
/* Toggle the LED when the GPT notification is called */
if (1U == toggleLed)
{
    toggleLed = 0;
    Dio_WriteChannel(DioConf_DioChannel_DioChannel_0, stateLed);
    stateLed = stateLed ^ 1U;
}
```

eMIOS Example Project: OCU (MCAL)

S32DS RTD 3.0.0 eMIOS Examples (MCAL)

- **Description:**
  - Initialize eMIOS channel to start counting. Using OCU driver, a notification will be generated upon each compare match and a while loop will run to blink an LED.
    - Notification function will toggle the PTB13 pin and will increase a 'count' variable by 1. The while loop will toggle the LED pin every 10 increments of the counter using DIO driver.
- **Dependent RTD functions:**
  - **Mcu_Init, Mcu_InitClock, Mcu_SetMode**:
    - Initialize the clock sources, the clock tree and to configure the clock gating of the peripherals.
  - **Port_Init, Mcl_Init**:
    - Initialize all pins using the Port driver. Initialize eMIOS common.
  - **Ocu_Init, Ocu_EnableNotification, Ocu_StartChannel, Dio_WriteChannel**:
    - Initialize the OCU driver, enable its notification and start channel. Use DIO driver to toggle LED on/off.
- **HW requirements:**
  - S32K3X8EVB-Q289 (schematic: SPF-54870 Rev.C)
- **For further information regarding the EMIOS OCU RTD driver, review the documentation in the following path (for default RTD installation):**
  - C:\NXP\S32DS.3.5\S32DS\software\PlatformSDK_S32K3\RTD\Ocu_TS_T40D34M30I0R0\doc

eMIOS Example Project: OCU (MCAL) (Continued)

S32DS RTD 3.0.0 eMIOS Examples (MCAL)

- **eMIOS OCU example:**
  - main.c:

Component	Description	Channel (EVB)
MCL	Initialize instance and master bus	eMIOS_0_CH0
OCU	Toggle pin state	eMIOS_0_CH1

```
/* Includes */
#include "Ocu.h"
#include "Port.h"
#include "Dio.h"
#include "Mcu.h"
#include "Platform.h"
#include "Mcl.h"

/* Local macros */
#define FXOSC_CLOCK_FREQ   16000000U

/* Global flag updated in IRQ */
static volatile uint8 toggleLed = 0U;

/* EMIOS notification periodically called.
Used to blink LED */
void OcuChannel_0_notification()
{
    static uint16 count = 0U;
    if(++count >= 10U)
    {
        toggleLed = 1U;
        count = 0;
    }
}
```

```
/* Initialize the MCU driver */
Mcu_Init(&Mcu_Config_VS_0);
/* Initialize the clock tree */
Mcu_InitClock(McuClockSettingConfig_0);
/* Apply a mode configuration */
Mcu_SetMode(McuModeSettingConf_0);
/* Initialize all pins using the Port driver */
Port_Init(&Port_Config_VS_0);\
/* Set eMios interrupt */
Platform_Init(NULL_PTR);
/* Initialize MCL driver */
Mcl_Init(&Mcl_Config_VS_0);
/* Initialize the high-level configuration structure of OCU driver */
Ocu_Init(&Ocu_Config_VS_0);
/* Enable eMIOS OCU channel notification */
Ocu_EnableNotification(OcuChannel_0);
/* Start eMIOS OCU Channel */
Ocu_StartChannel(OcuChannel_0);
while (1) {
    /* Toggle the LED when the OCU notification is called */
    if (1U == toggleLed) {
        toggleLed = 0;
        Dio_WriteChannel(DioConf_DioChannel_DioChannel_0, stateLed);
        stateLed = stateLed ^ 1U;
    }
}
```

eMIOS Example Project: PWM (MCAL)

S32DS RTD 3.0.0 eMIOS Examples (MCAL)

- **Description:**
  - PWM example using eMIOS to turn on LED and regulate its brightness.
- **Dependent RTD functions:**
  - **Mcu_Init, Mcu_InitClock, Mcu_SetMode**:
    - Initialize the clock sources, the clock tree and to configure the clock gating of the peripherals.
  - **Siul2_Port_Ip_Init**:
    - Initialize all pins using the Siul2_Port driver.
  - **Mcl_Init**:
    - Initialize eMIOS common.
  - **Pwm_Init, Mcl_Emios_SetCounterBusPeriod, Pwm_SetDutyCycle**:
    - Initialize the PWM driver, change the period and duty cycle to control the LED brightness and blink LED.
- **HW requirements:**
  - S32K3X8EVB-Q289 (schematic: SPF-54870 Rev.C)
- **For further information regarding the EMIOS PWM RTD driver, review the documentation in the following path (for default RTD installation):**
  - C:\NXP\S32DS.3.5\S32DS\software\PlatformSDK_S32K3\RTD\Pwm_TS_T40D34M30I0R0\doc

eMIOS Example Project: PWM (MCAL) (Continued)

S32DS RTD 3.0.0 eMIOS Examples (MCAL)

- **eMIOS PWM example:**
  - main.c:

Component	Description	Channel (EVB)
MCL	Initialize instance and master bus	eMIOS_1_CH23
PWM	Generate OPWMB PWM signal	eMIOS_1_CH5

```c
/* Includes */
#include "Pwm.h"
#include "Mcu.h"
#include "Siul2_Port_Ip.h"
#include "Mcl.h"

/* Local macros */
#define NUM_BLINK_LED  (uint32)10U
#define DELAY_TIMER    (uint32)5000000U
#define MCL_EMIOS_1_CH_23 (uint16)23U

/* Local functions */
void TestDelay(uint32 delay)
{
    static volatile uint32 DelayTimer = 0;
    while(DelayTimer<delay)
    {
        DelayTimer++;
    }
    DelayTimer=0;
}
```

```c
/* Initialize the Mcu driver */
Mcu_Init(&Mcu_Config_VS_0);
/* Initialize the clock tree */
Mcu_InitClock(McuClockSettingConfig_0);
/* Apply a mode configuration */
Mcu_SetMode(McuModeSettingConf_0);
/* Initialize all pins using the Siul2_Port driver */
Siul2_Port_Ip_Init(NUM_OF_CONFIGURED_PINS, g_pin_mux_InitConfigArr_VS_0);
/* Initialize MCL driver */
Mcl_Init(&Mcl_Config_VS_0);
/* Initialize Pwm driver , after that Led on*/
Pwm_Init(&Pwm_Config_VS_0);
/* Set new period for all channels used external counter bus */
Mcl_Emios_SetCounterBusPeriod(MCL_EMIOS_1_CH_23, 0x9000U, FALSE);
/* Led on when duty cycle is 100% */
Pwm_SetDutyCycle(EMIOS_1_CH_5, 0x8000);
TestDelay(DELAY_TIMER);
/* Led off when duty cycle is 0% */
Pwm_SetDutyCycle(EMIOS_1_CH_5, 0x0000);
TestDelay(DELAY_TIMER);
/* De-Initialize Pwm driver */
Pwm_DeInit();
```

eMIOS Example Project: ICU (MCAL)

S32DS RTD 3.0.0 eMIOS Examples (MCAL)

- **Description:**
  - GPIO trigger will generate edges to ICU input pins. Interrupts will be generated for the ICU channels and the user function will be called to blink the LED.
- **Dependent RTD functions:**
  - **Mcu_Init, Mcu_InitClock, Mcu_SetMode**:
    - Initialize the clock sources, the clock tree and to configure the clock gating of the peripherals.
  - **Platform_Init, Port_Init**:
    - Initialize platform driver. Initialize all pins using the Port driver.
  - **Platform_InstallIrqHandler, Platform_SetIrq, Mcl_Init**:
    - Install eMIOS ISR. Initialize eMIOS common.
  - **Icu_Init, Icu_EnableEdgeDetection, Icu_EnableNotification, Dio_WriteChannel**:
    - Initialize, enable edge detection and notification of ICU driver. Use DIO to generate input for ICU channel and toggle LED.
- **HW requirements:**
  - S32K3X8EVB-Q289 (schematic: SPF-54870 Rev.C)
- **For further information regarding the EMIOS ICU RTD driver, review the documentation in the following path (for default RTD installation):**
  - C:\NXP\S32DS.3.5\S32DS\software\PlatformSDK_S32K3\RTD\Icu_TS_T40D34M30I0R0\doc

eMIOS Example Project: ICU (MCAL) (Continued)

S32DS RTD 3.0.0 eMIOS Examples (MCAL)

- **eMIOS ICU example:**
  - main.c:

Component	Description	Channel (EVB)
MCL	Initialize instance and master bus	eMIOS_0_CH23
ICU	Both edges detection	eMIOS_0_CH2

```
/* Global functions */
extern ISR(EMIOS0_5_IRQ);
/* Blink LED using user function */
void User_EdgeDetect0(void) {
    /* Increment IRQ counter */
    UserCountIrqCH0++;
    if(UserCountIrqCH0 % 2 == 0) {
Dio_WriteChannel(DioConf_DioChannel_Digita
l_Output_LED_D39, STD_HIGH);
    }
    else {
Dio_WriteChannel(DioConf_DioChannel_Digita
l_Output_LED_D39, STD_LOW);
    }
}
/* Initialize the Mcu driver */
Mcu_Init(&Mcu_Config_VS_0);
/* Initialize the clock tree */
Mcu_InitClock(McuClockSettingConfig_0);
/* Apply a mode configuration */
Mcu_SetMode(McuModeSettingConf_0);
/* Initialize Platform driver */
Platform_Init(NULL_PTR);
```

```
/* Install EMIOS ISR */
Platform_InstallIrqHandler(EMIOS0_5_IRQn, &EMIOS0_5_IRQ, NULL_PTR);
Platform_SetIrq(EMIOS0_5_IRQn,TRUE);
/* Initialize all pins using the Port driver */
Port_Init(NULL_PTR);
/* Initialize all pins using the Port driver */
Mcl_Init(NULL_PTR);
/* Initialize the ICU driver */
Icu_Init(NULL_PTR);
/* Enable ICU edge detection and notification */
Icu_EnableEdgeDetection(IcuChannel_0);
Icu_EnableNotification(IcuChannel_0);
/* Generate input for ICU channel */
while (BlinkCount < NUM_MAX_BLINK) {
    if(BlinkCount % 2 == 0) {
        Dio_WriteChannel(DioConf_DioChannel_Port_pin_InputForIcuChannel0, STD_HIGH);
        delay();
    } else {
        Dio_WriteChannel(DioConf_DioChannel_Port_pin_InputForIcuChannel0, STD_LOW);
        delay();
    }
    BlinkCount++;
}
```



nxp.com

| **Public** | NXP, and the NXP logo are trademarks of NXP B.V. All other product or service names are the property of their respective owners. © 2024 NXP B.V.